

Deep Learning for Text Analytics



The Transformer

Stefan Lessmann [stefan.lessmann@hu-berlin.de]



RECAP: 2018 – The NLP ImageNet “Moment”

RNN-based language models dethroned

2018 – when NLP ImageNet became reality

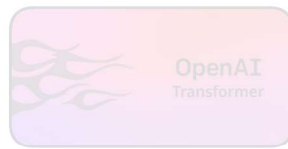
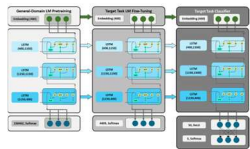
ULMFiT

ELMo

GPT

BERT

GPTv2



Jan 2018

Feb 2018

Jan 2018

Oct 2018

Feb 2019

1 GPU day

~ 42 GPU days

240 GPU days

256 TPU days

~2048 TPU days

Based on transformers



A Fifteen-Page Paper Revolutionizing Deep Learning and AI

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best

Annotated paper by Havard NLP group
<http://nlp.seas.harvard.edu/2018/04/03/attention.html>



Ashish Vaswani

Startup
Verified email at fastmail.com
Deep Learning

FOLLOW

TITLE	CITED BY	YEAR
Attention is all you need A Vaswani, N Shazeer, N Parmar, J Uszkoreit, L Jones, AN Gomez, ... Advances in neural information processing systems 30	127769	2017
Relational inductive biases, deep learning, and graph networks PW Battaglia, JB Hamrick, V Bapst, A Sanchez-Gonzalez, V Zambaldi, ... arXiv preprint arXiv:1806.01261	3547	2018
Self-attention with relative position representations P Shaw, J Uszkoreit, A Vaswani arXiv preprint arXiv:1803.02155	2406	2018



Ashish Vaswani · 2nd
Co-founder and CEO at Essential AI
San Francisco, California, United States · [Contact info](#)
21,050 followers · 500+ connections
[Joseph de Castelneau](#), [Amir Thapa Magar](#), [M. Sc.](#), and 9 other mutual connections

[Connect](#) [Message](#) [More](#)

Experience

 Co-Founder and CEO
Essential AI · Full-time
Dec 2022 - Present · 1 yr 7 mos
San Francisco Bay Area · On-site
Pushing the frontier of human machine partnership

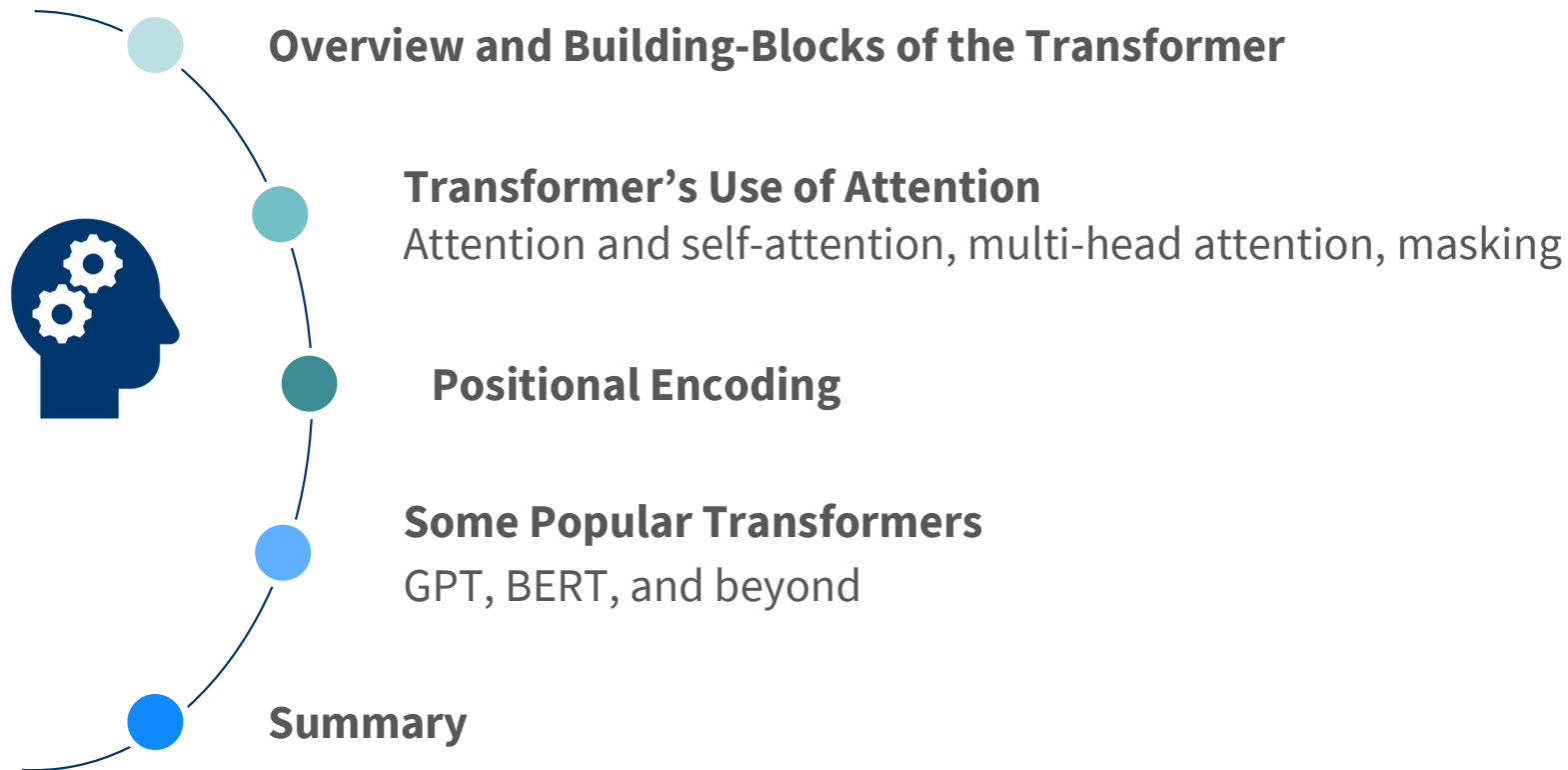
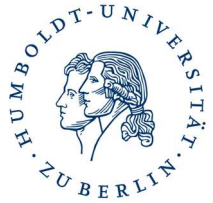
 Co-Founder and Chief Scientist
Adept AI Labs · Full-time
Jan 2022 - Nov 2022 · 11 mos
San Francisco Bay Area

 Staff Research Scientist
Google Brain
Jul 2016 - Nov 2021 · 5 yrs 5 mos
Mountain View
At Google Brain

 Computer Scientist
Information Sciences Institute
Jul 2014 - Jun 2016 · 2 yrs
Marina Del Rey

 Graduate Research Assistant
University of Southern California
May 2008 - May 2014 · 6 yrs 1 mo

Agenda





Overview and Building-Blocks of the Transformer

The Transformer: A Fifteen-Page Paper Revolutionizing AI

Key building blocks

Attention Is All You Need

NIPS 2017, pp. 6000-6010.

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

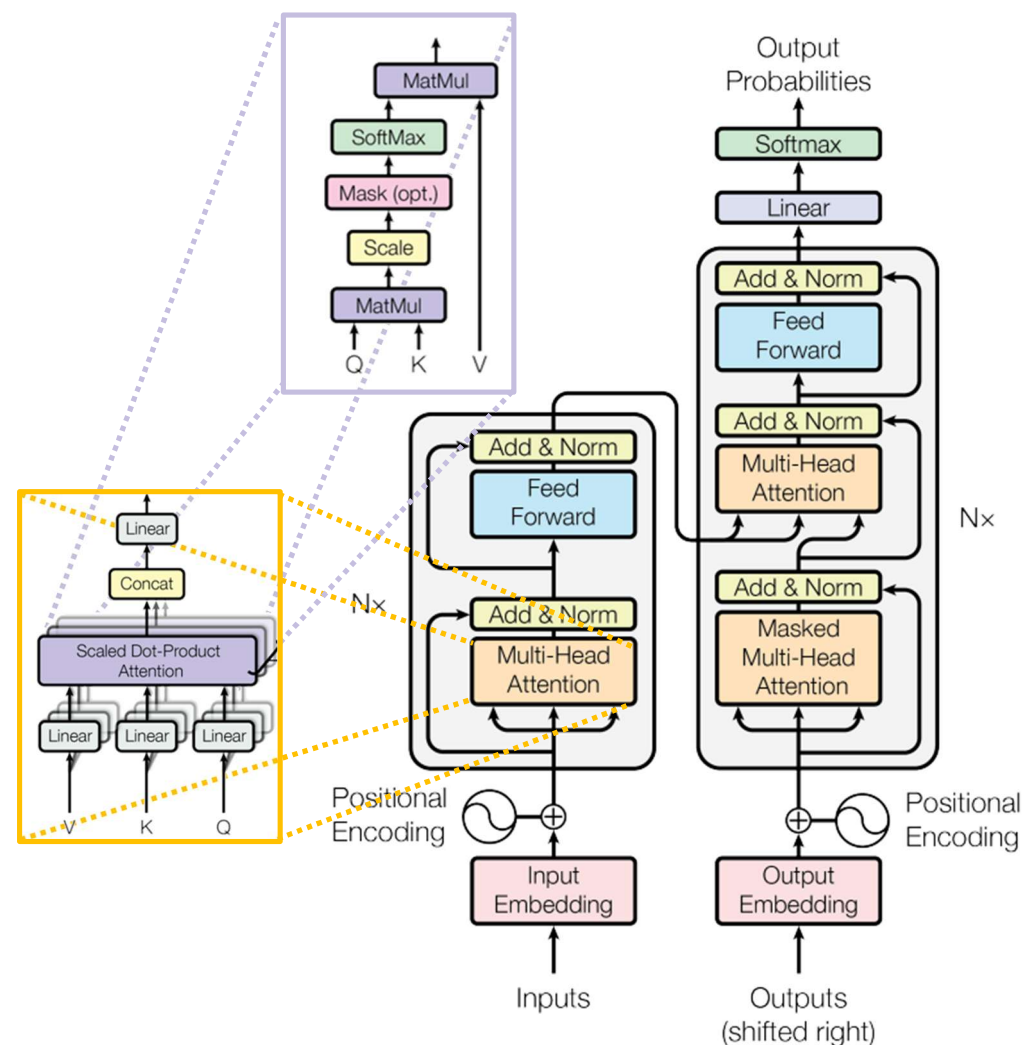
Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions

Annotated paper by Harvard NLP group
<http://nlp.seas.harvard.edu/2018/04/03/attention.html>



The Transformer: A Fifteen-Page Paper Revolutionizing AI

Key building blocks

■ Introduced for machine translation

- Multi-class classifier output layer
- Predict words from a target vocabulary

■ Embeddings to encode text data

■ Sequence-to-sequence architecture

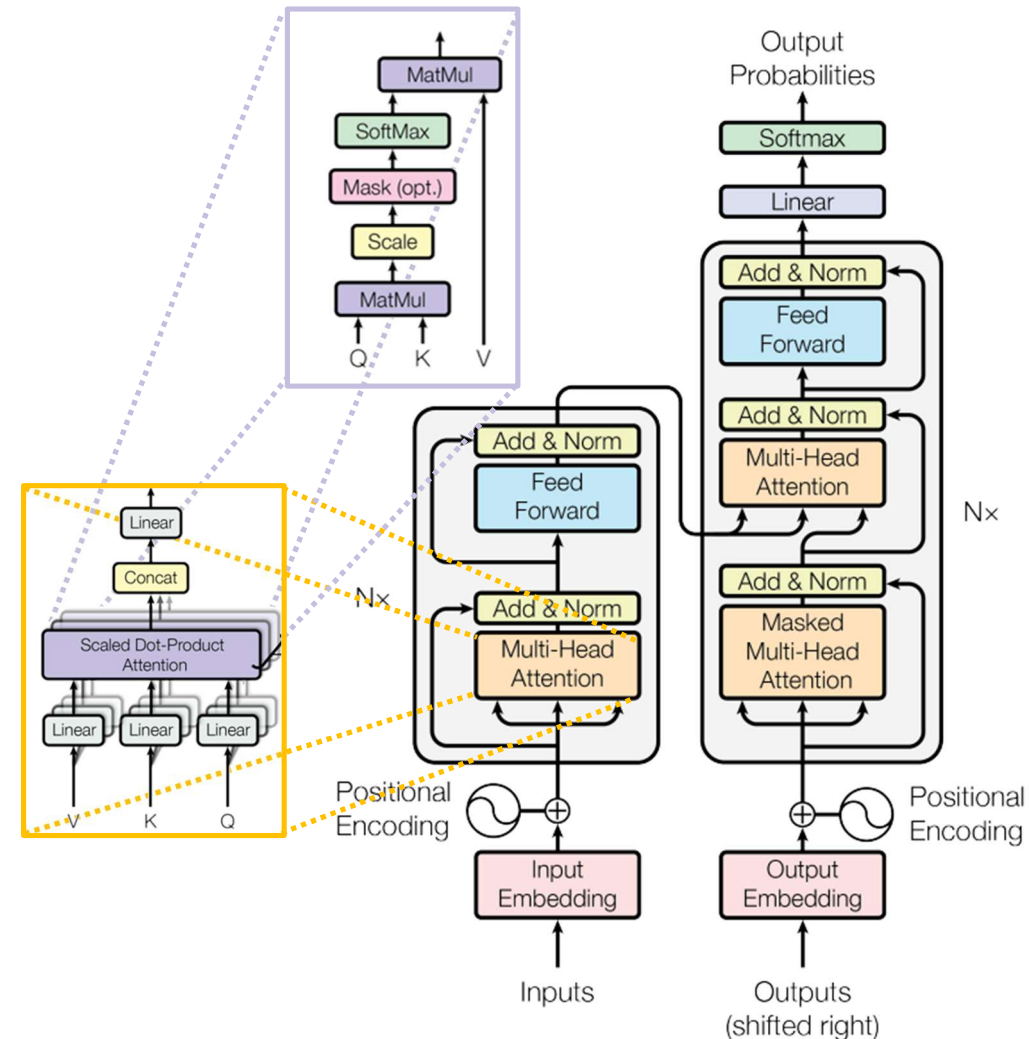
- Encoder produces an embedding
- Decoder produces an output based on encoder embedding and its own inputs (if any)

■ Depth by stacking similar *blocks*

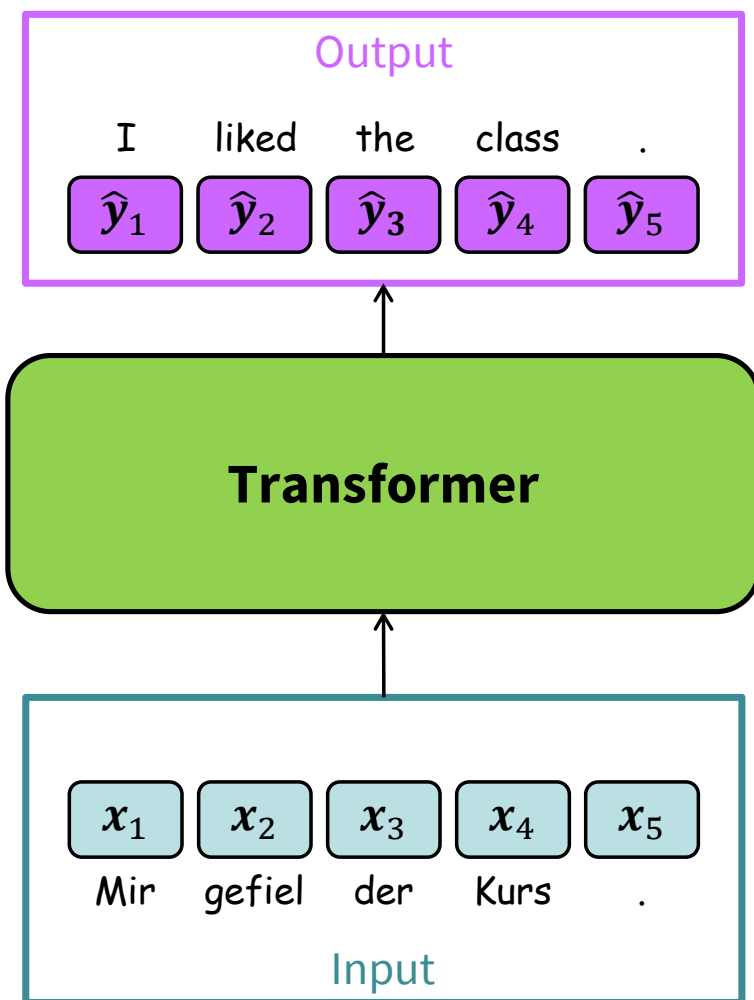
■ Transformer block

- Dense (i.e., FFN) layer
- Residual connection
- Batch normalization
- Multi-head attention

■ Positional encoding



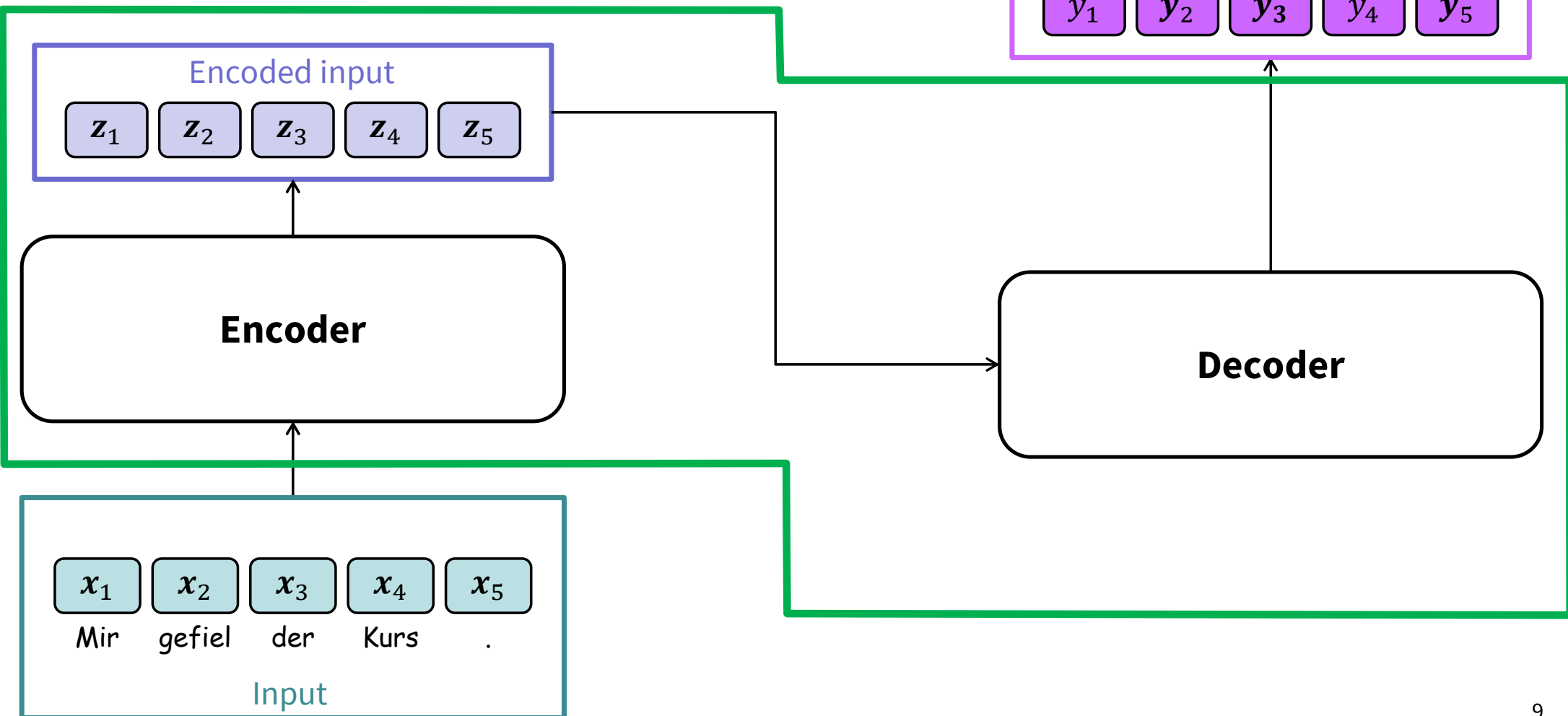
A Transformer Transforms Input Sequences to Output Sequences



Most competitive neural sequence transduction models have an encoder-decoder structure [5, 2, 35]. Here, the encoder maps an input sequence of symbol representations $(x_1, \dots, x_n)$ to a sequence of continuous representations $z = (z_1, \dots, z_n)$. Given z , the decoder then generates an output sequence $(y_1, \dots, y_m)$ of symbols one element at a time. At each step the model is auto-regressive [10], consuming the previously generated symbols as additional input when generating the next. [Vaswani et al. (2017) p.2]

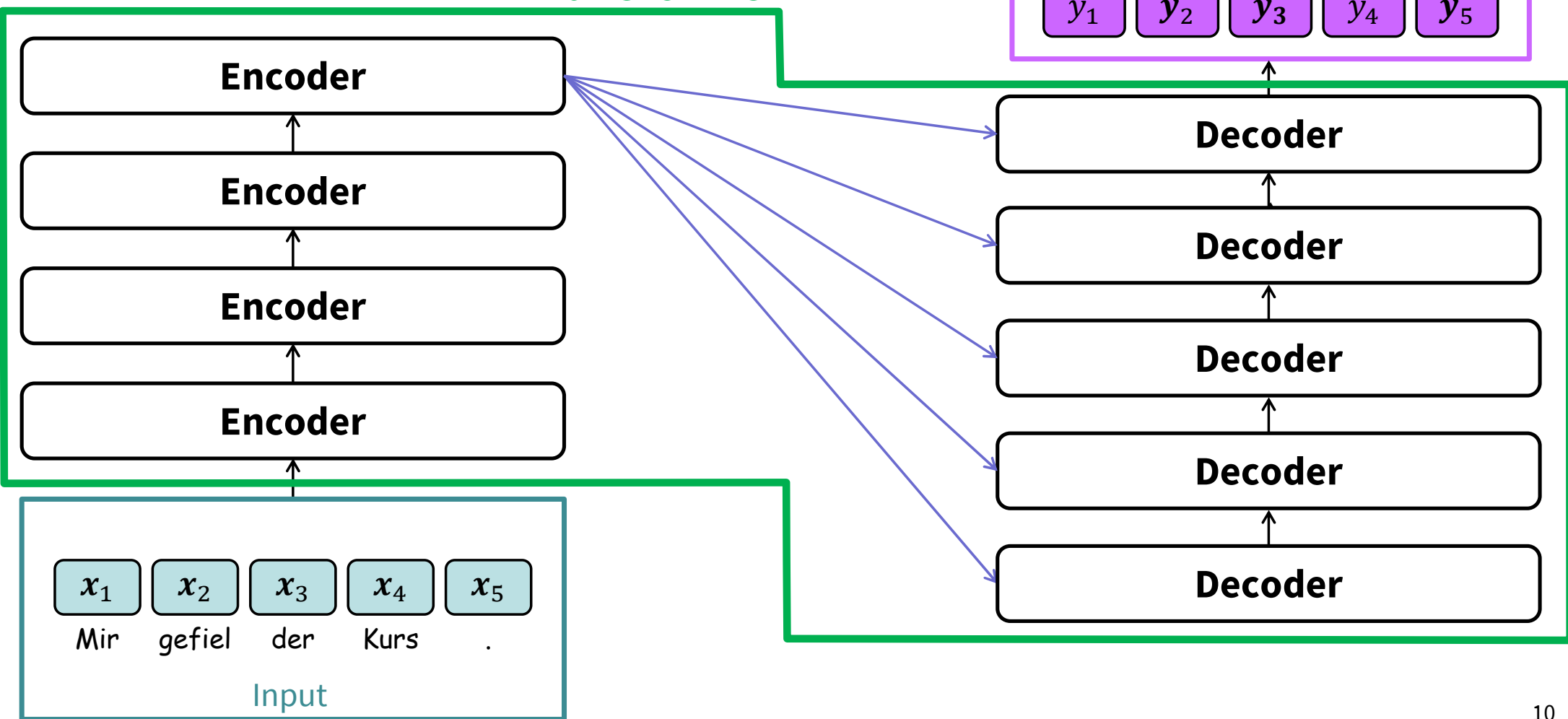
Sequence-to-Sequence (Seq2Seq) Revisited

Transformer



The Transformer is a Deep Seq2Seq Model

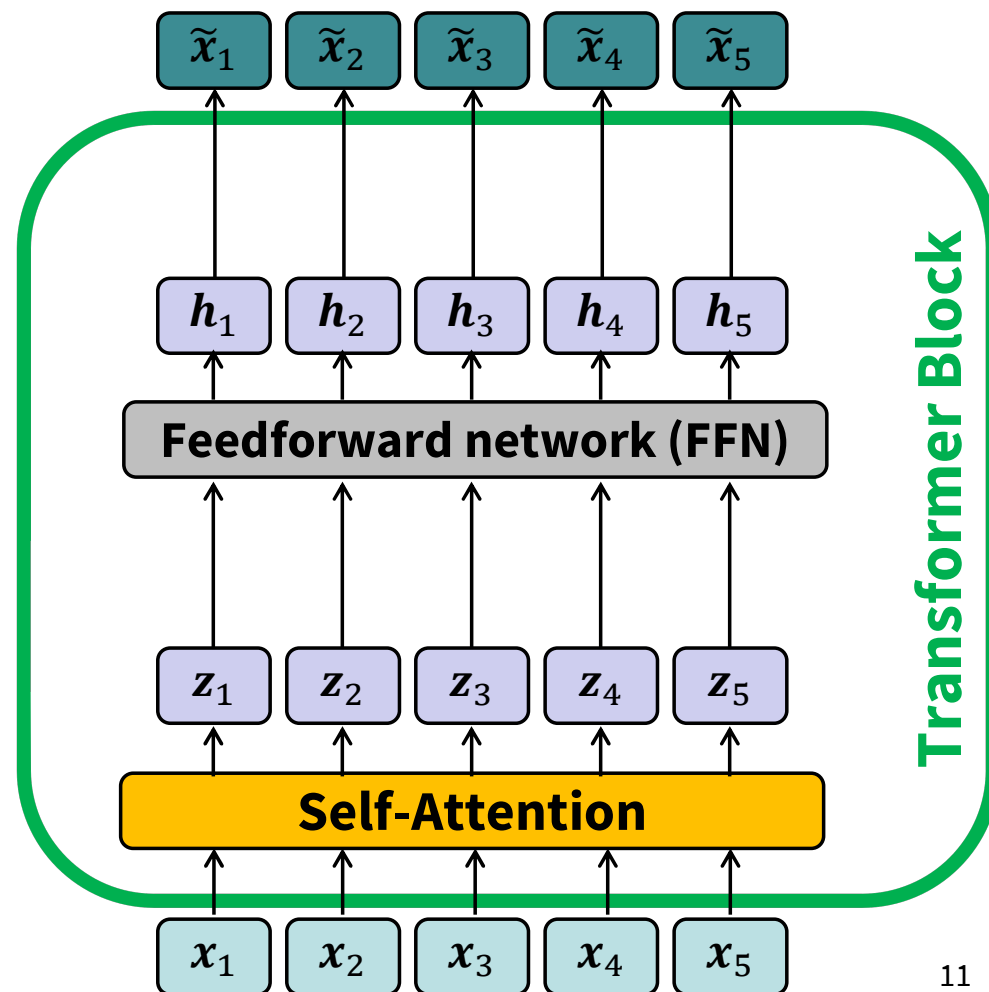
Transformer



Encoder & Decoder Layers Consist of Transformer Blocks

Two sublayers in each block: **attention** and feedforward network (FFN)

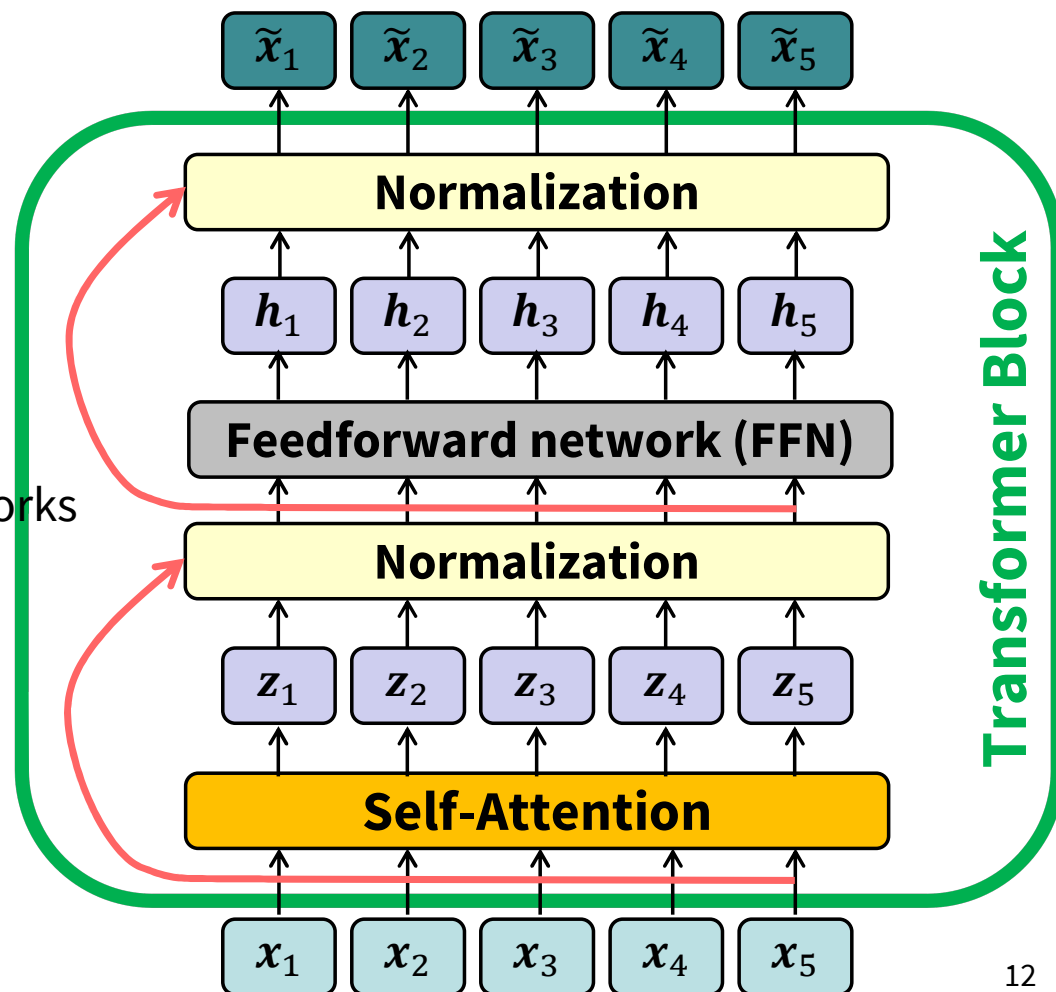
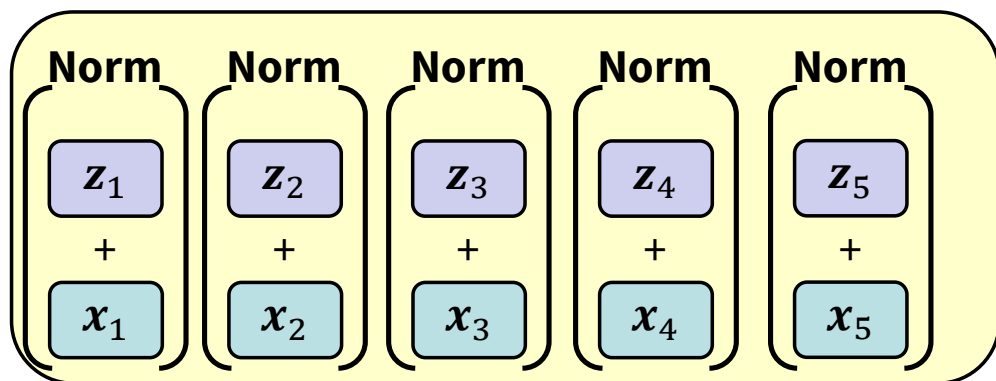
- **Input sequence of T tokens** $X = (x_1, \dots, x_T) \in \mathbb{R}^{T \times d}$ with token embedding $x_t \in \mathbb{R}^d$
- **Self-Attention layer produces a transformed sequence** $Z = (z_1, \dots, z_T) \in \mathbb{R}^{T \times d}$
 - Capture dependencies among tokens
 - Parallel processing of sequence tokens
- **Position-wise application of FFN**
 - Each token $z_t \in \mathbb{R}^d$ as individual input
 - Adding nonlinearity (attention is a linear operation)
 - Same weights for all tokens enable parallel processing
- **Transformer block sustains dimensionality**
 - Output sequence has same shape as input sequence
 - Every token is transformed (embedded)
- **Block's output** $\tilde{X} = (\tilde{x}_1, \dots, \tilde{x}_T) \in \mathbb{R}^{T \times d}$ **becomes next block's input**

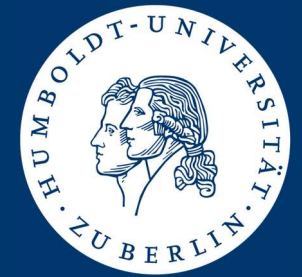


Encoder & Decoder Layers Consist of Transformer Blocks

Residual connections and layer normalization around each sublayer

- **Residual connections** let data bypass a sublayer
- **Output of sublayer becomes**
 - $\text{LayerNorm}(X + \text{SubLayer}(X))$
 - with $\text{SubLayer}()$ being either attention or FFN
- **Layer normalization**
 - Stabilize the learning and enable deeper networks
 - Standardizes each token independently





Transformer's Use of Attention

Attention and self-attention, multi-head attention,
masking

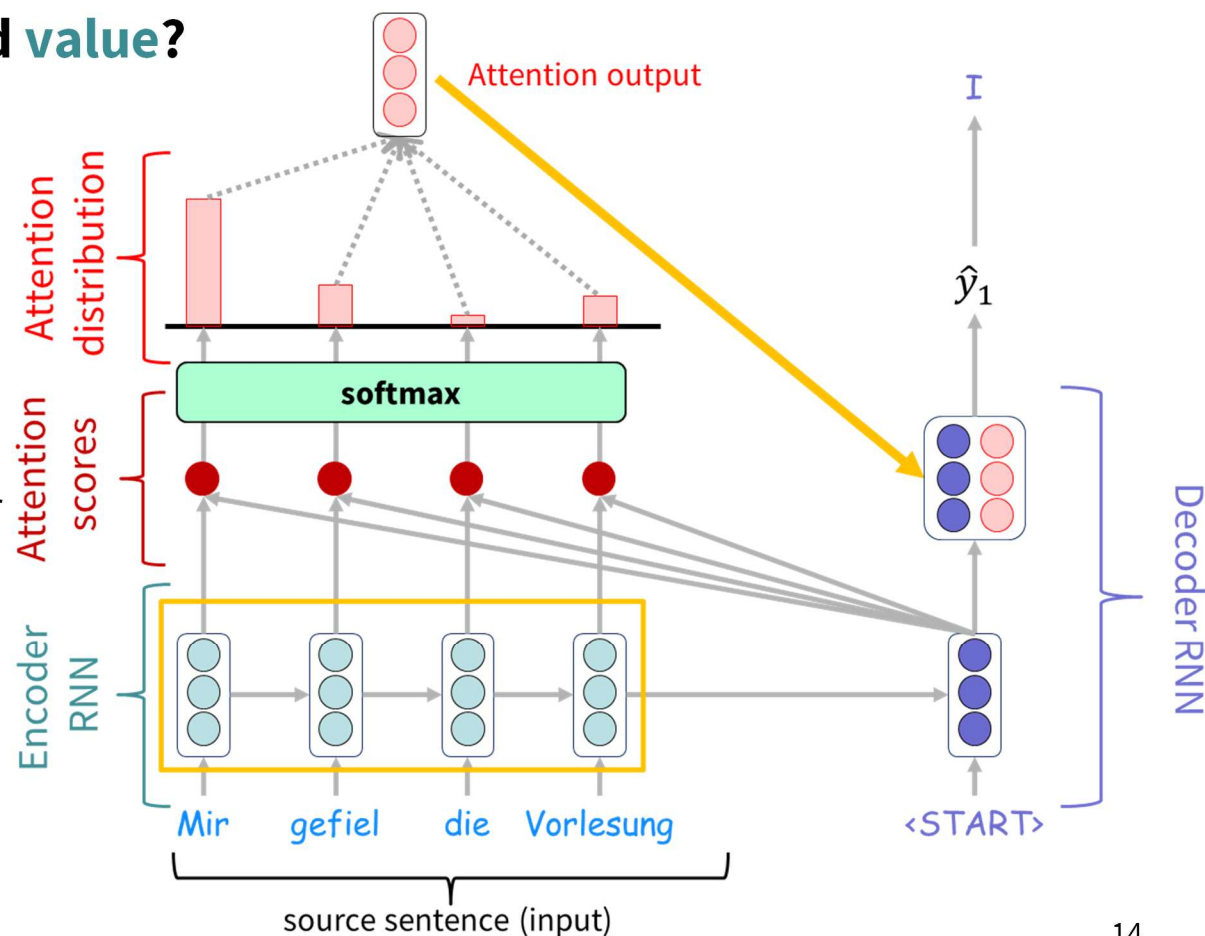
RECAP: Attention Mechanism

Attention pooling facilitates aggregating data from many (key, value) pairs

■ What specifically are **query**, **key**, and **value**?

■ Recall machine translation setting

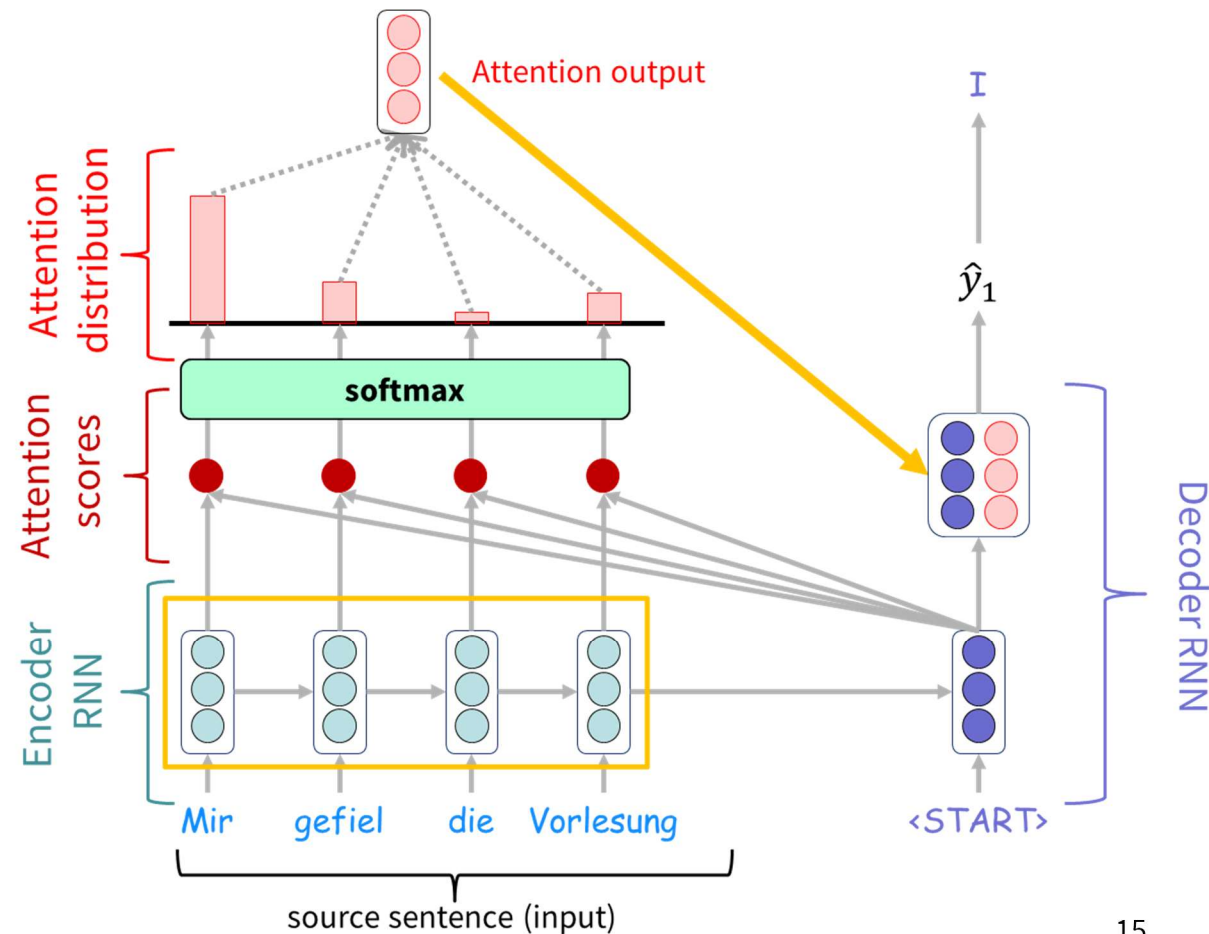
- The query is current state of the decoder
- The **keys** are encoder hidden states
 - From the **query** and the **keys**, we compute **attention weights**
 - They are also called alignment scores as they capture which encoder tokens is most relevant for the current context of the decoder
- The **values** are encoder hidden states, too
 - Using **attention weights**, we compute a weighted sum over the encoder states
 - Decoder uses this context vector to generate its next output



RECAP: Attention Mechanism

Attention pooling facilitates aggregating data from many (key, value) pairs

The attention mechanism computes a context vector as a weighted sum of the values (encoder states), using weights that capture how well the query (decoder state) matches each key (also encoder states).



Self-Attention

The transformer uses self-attention to avoid recurrent computations

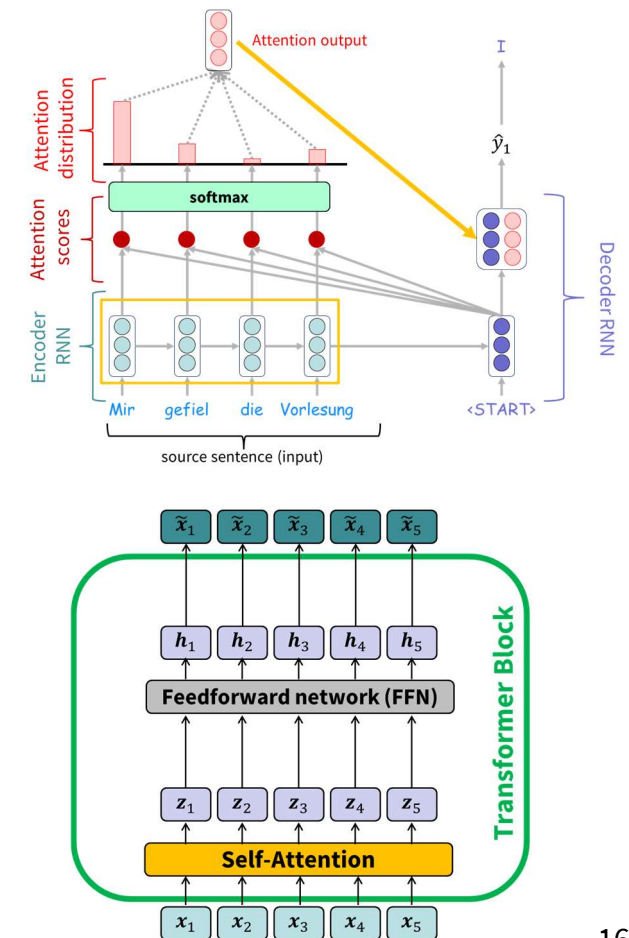
■ How is attention „all we need“?

■ Encode-Decoder Attention is a form of external attention

- Decoder uses attention over the encoder's output to compute alignment scores
- This form of attention depends on both the source sequence (through encoder states) and the target sequence (through decoder states)

■ Self-Attention is applied internally (over same sequence)

- Every element in the sequence generates query, key, value vectors
- It uses these vectors to attend to every other element of the (same) sequence, independent of external sequences
- Removes sequential processing as input positions can be treated independently for each layer



Self-Attention in Action

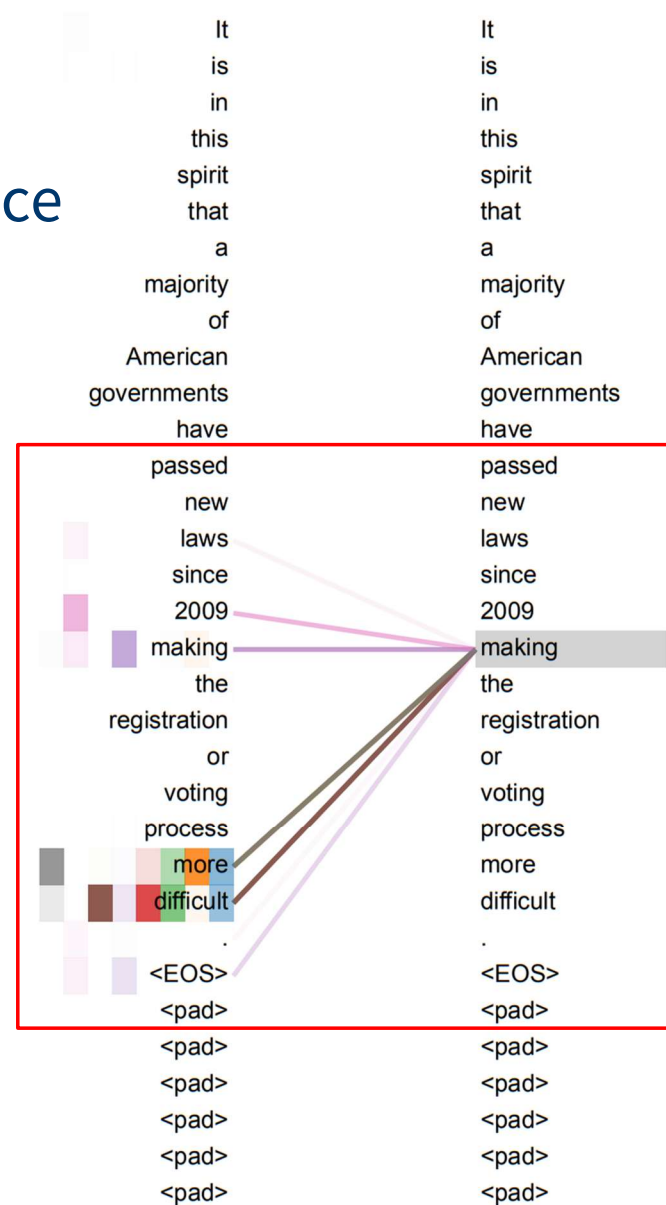
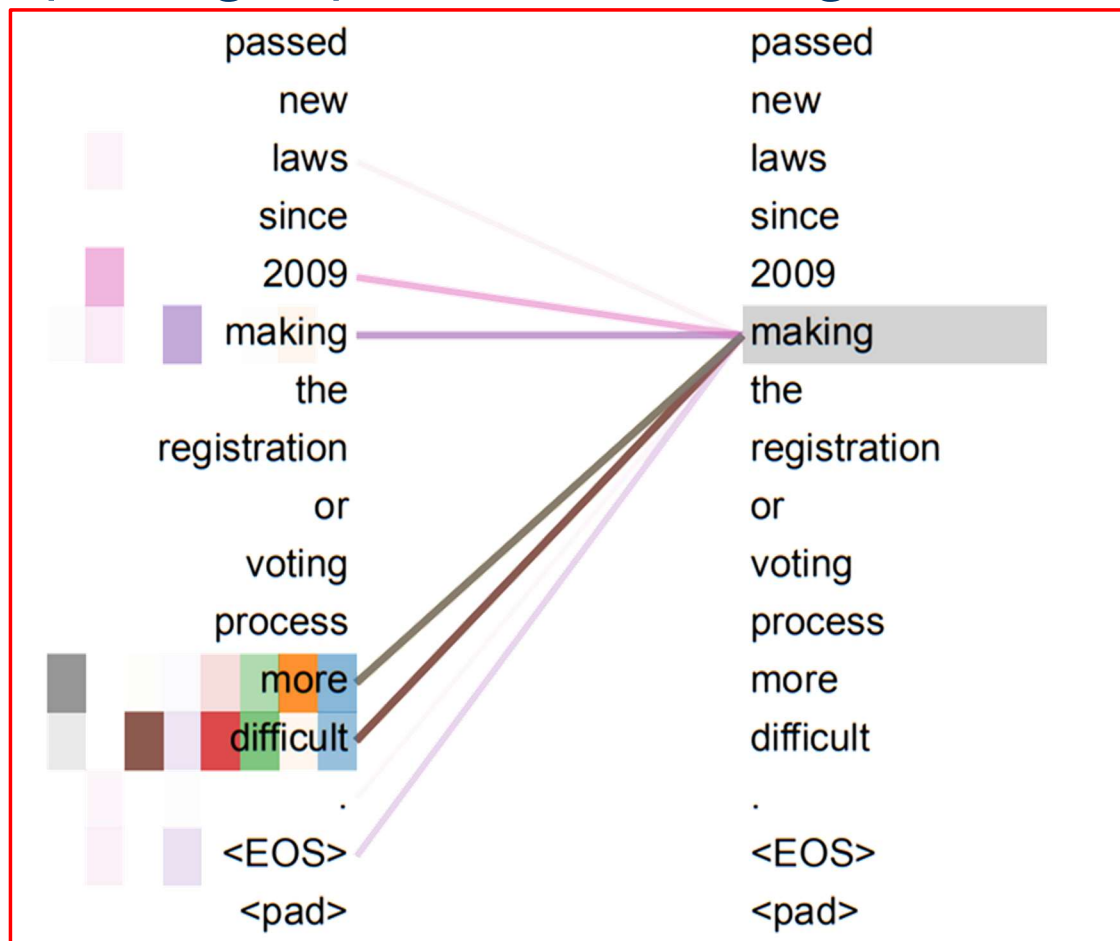
Which tokens *attend* to another?

The lecture of today was really hard to follow !
 x_1 x_2 x_3 x_4 x_5 x_6 x_8 x_9 x_7 x_{10}

x_1 x_2 x_3 x_4 x_5 x_6 x_8 x_9 x_7 x_{10}
The lecture of today was really hard to follow !

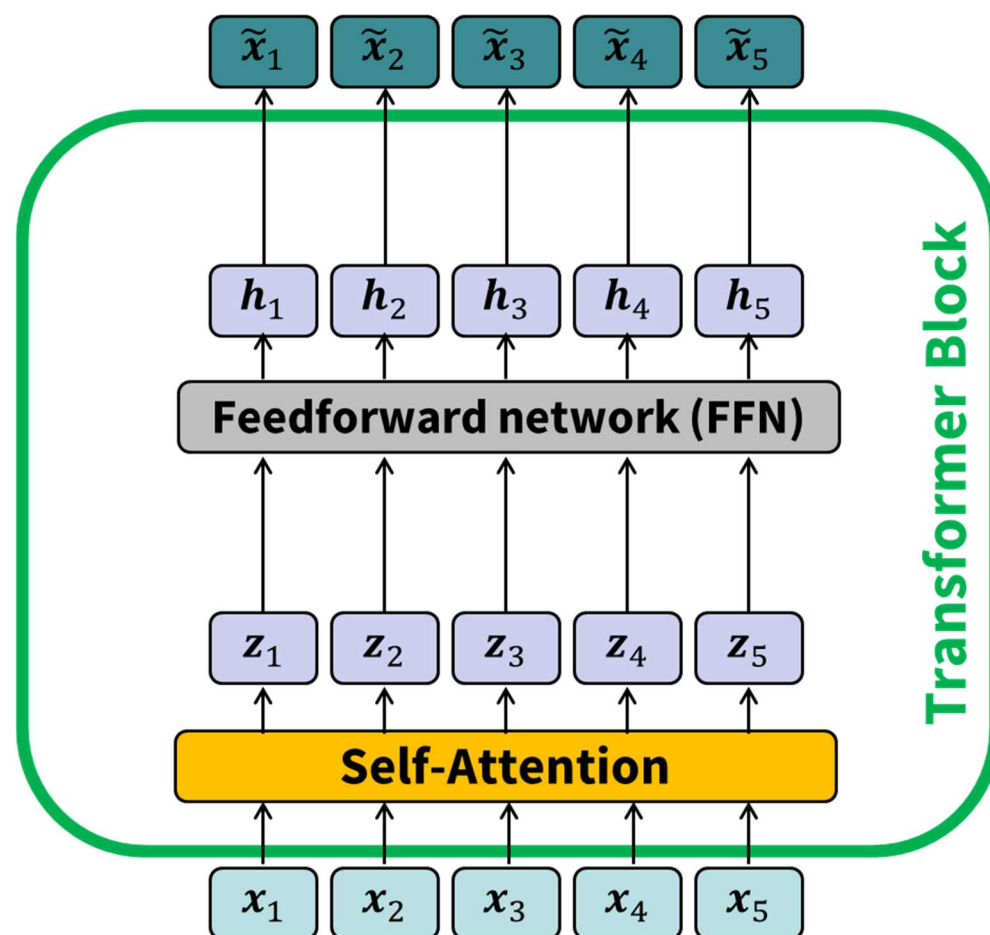
Self-Attention in Action

Capturing dependencies among tokens of a sequence



Self-Attention in Detail

- **Input sequence of T tokens**
 $X = (x_1, \dots, x_T) \in \mathbb{R}^{T \times d}$ **with**
token embedding $x_t \in \mathbb{R}^d$
- **Derive query, key, and value vectors from each token**
 - $q_t, k_t, v_t \in \mathbb{R}^{\tilde{d}}$
 - Vaswani et al. (2017) set $\tilde{d} = 64$
 - Yet another meta-parameter



Self-Attention in Detail

Introduce weight matrices

■ Input sequence of T tokens

$X = (x_1, \dots, x_T) \in \mathbb{R}^{T \times d}$ with token embedding $x_t \in \mathbb{R}^d$

■ Derive query, key, and value vectors from each token

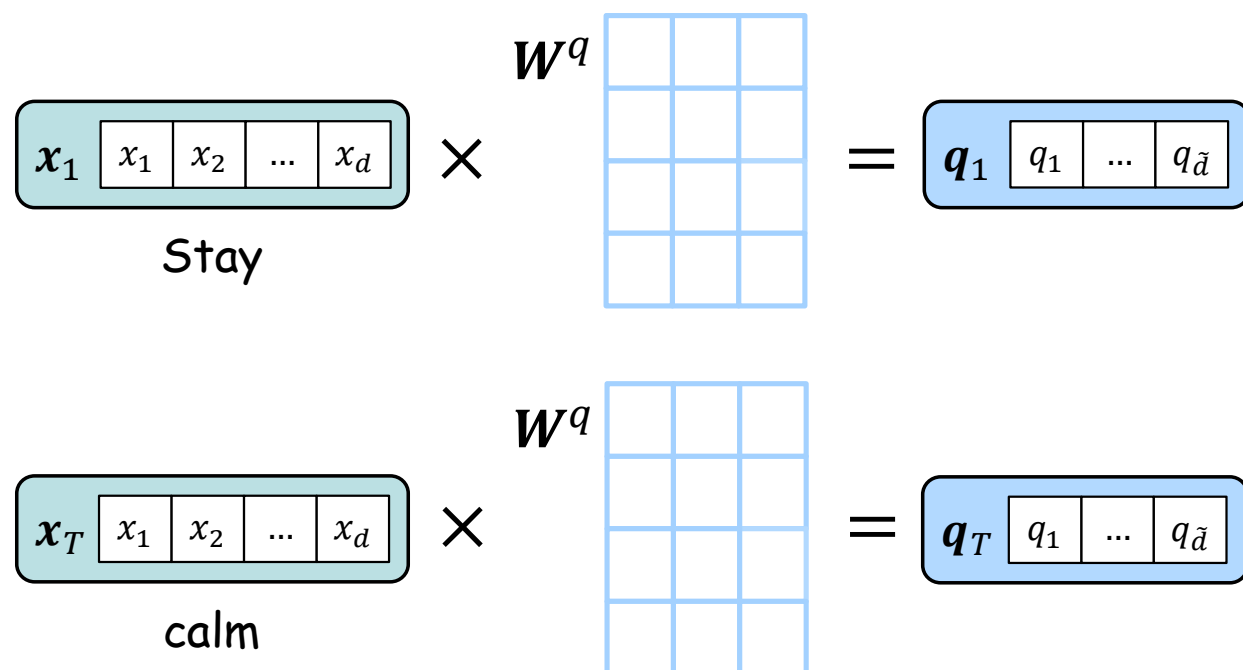
□ $q_t, k_t, v_t \in \mathbb{R}^{\tilde{d}}$

– Vaswani et al. (2017) set $\tilde{d} = 64$

– Yet another meta-parameter

□ Compute as $q_t = x_t$

with $W^q \in \mathbb{R}^{d \times \tilde{d}}$



Self-Attention in Detail

Introduce weight matrices

■ Input sequence of T tokens

$X = (x_1, \dots, x_T) \in \mathbb{R}^{T \times d}$ with token embedding $x_t \in \mathbb{R}^d$

■ Derive query, key, and value vectors from each token

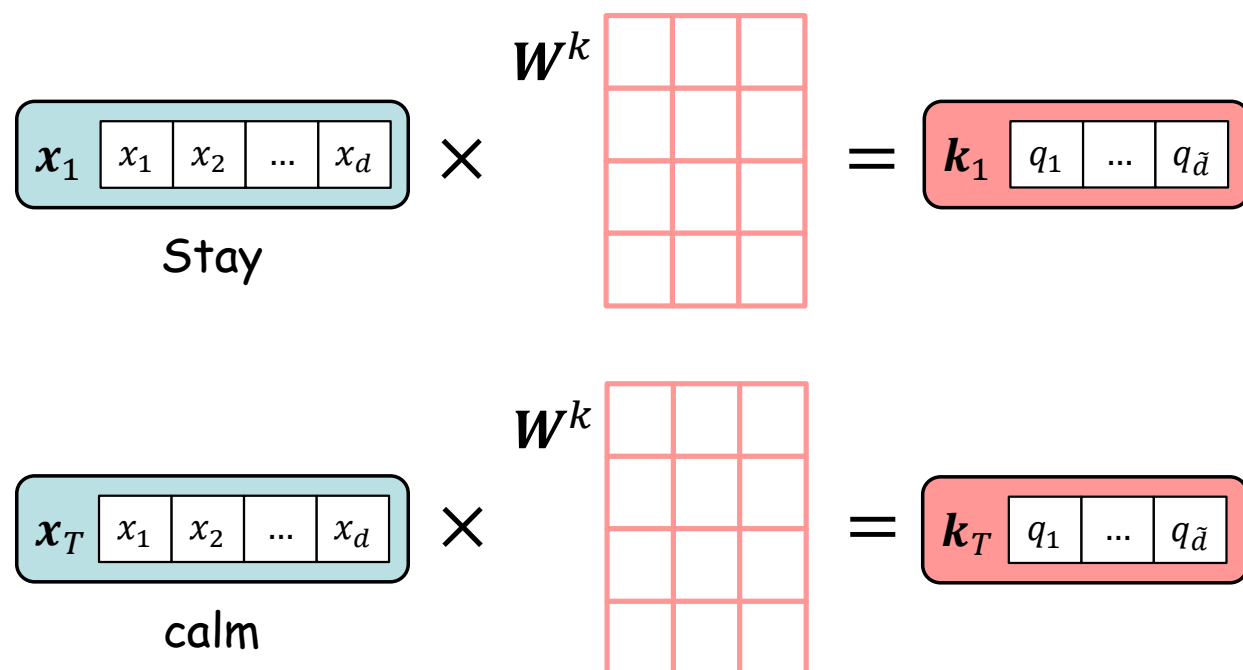
□ $q_t, k_t, v_t \in \mathbb{R}^{\tilde{d}}$

– Vaswani et al. (2017) set $\tilde{d} = 64$

– Yet another meta-parameter

□ Compute as $q_t = x_t W^q$, $k_t = x_t W^k$,

with $W^q, W^k \in \mathbb{R}^{d \times \tilde{d}}$



Self-Attention in Detail

Introduce weight matrices

■ Input sequence of T tokens

$X = (x_1, \dots, x_T) \in \mathbb{R}^{T \times d}$ with token embedding $x_t \in \mathbb{R}^d$

■ Derive query, key, and value vectors from each token

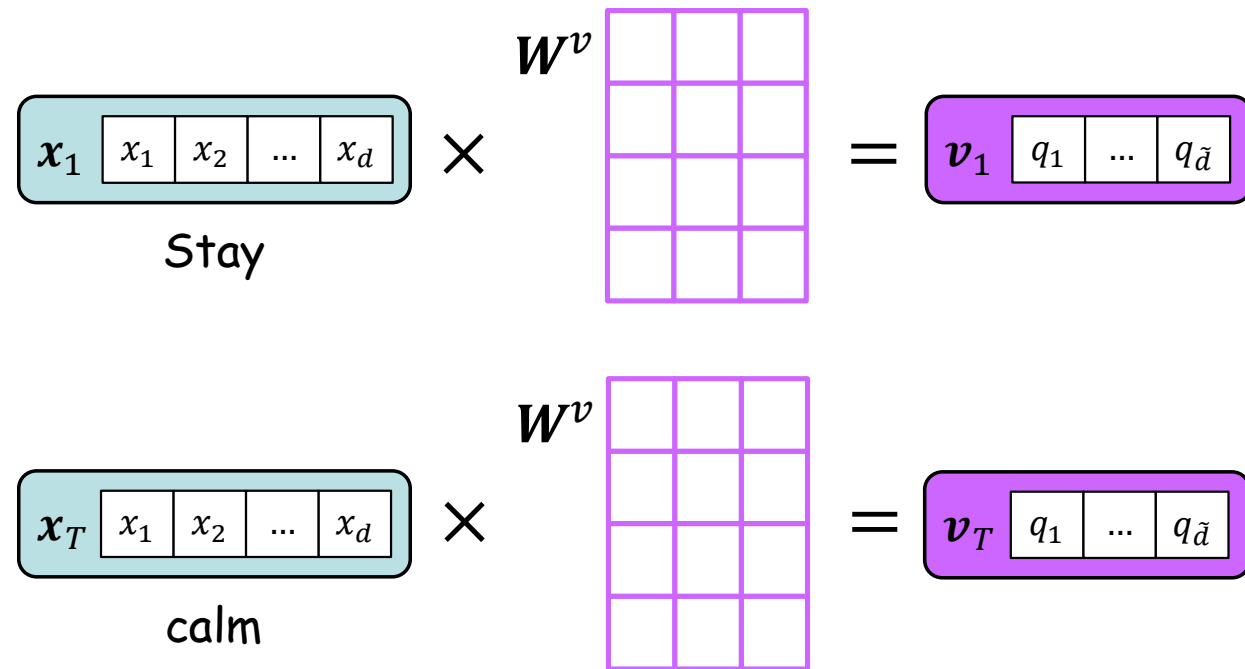
□ $q_t, k_t, v_t \in \mathbb{R}^{\tilde{d}}$

– Vaswani et al. (2017) set $\tilde{d} = 64$

– Yet another meta-parameter

□ Compute as $q_t = x_t W^q$, $k_t = x_t W^k$,
 $v_t = x_t W^v$

with $W^q, W^k, W^v \in \mathbb{R}^{d \times \tilde{d}}$



Self-Attention in Detail

Introduce weight matrices

■ Input sequence of T tokens

$X = (x_1, \dots, x_T) \in \mathbb{R}^{T \times d}$ with token embedding $x_t \in \mathbb{R}^d$

■ Derive query, key, and value vectors from each token

□ $q_t, k_t, v_t \in \mathbb{R}^{\tilde{d}}$

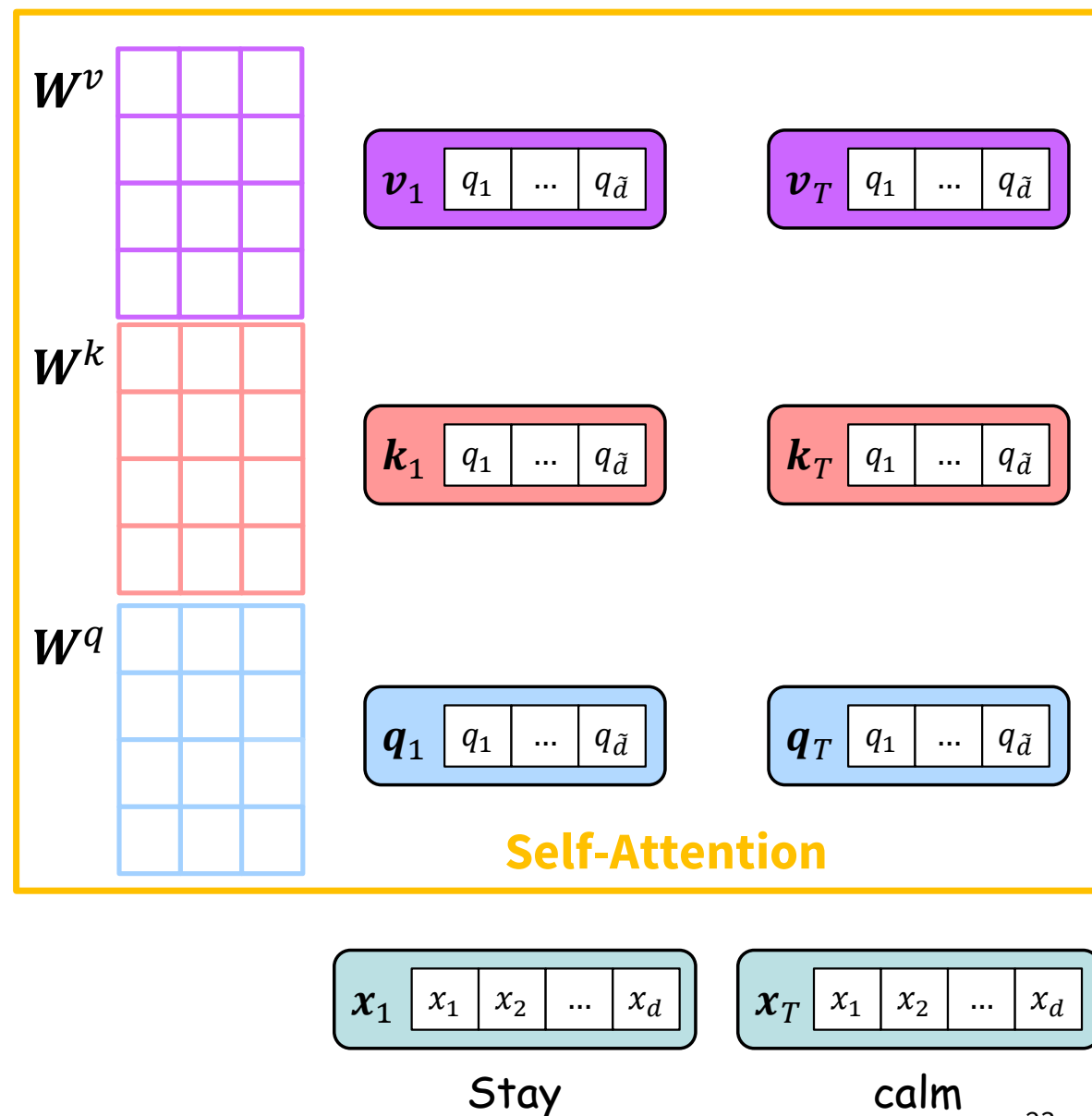
– Vaswani et al. (2017) set $\tilde{d} = 64$

– Yet another meta-parameter

□ Compute as $q_t = x_t W^q$, $k_t = x_t W^k$, $v_t = x_t W^v$

with $W^q, W^k, W^v \in \mathbb{R}^{d \times \tilde{d}}$

□ The matrices are trainable weights

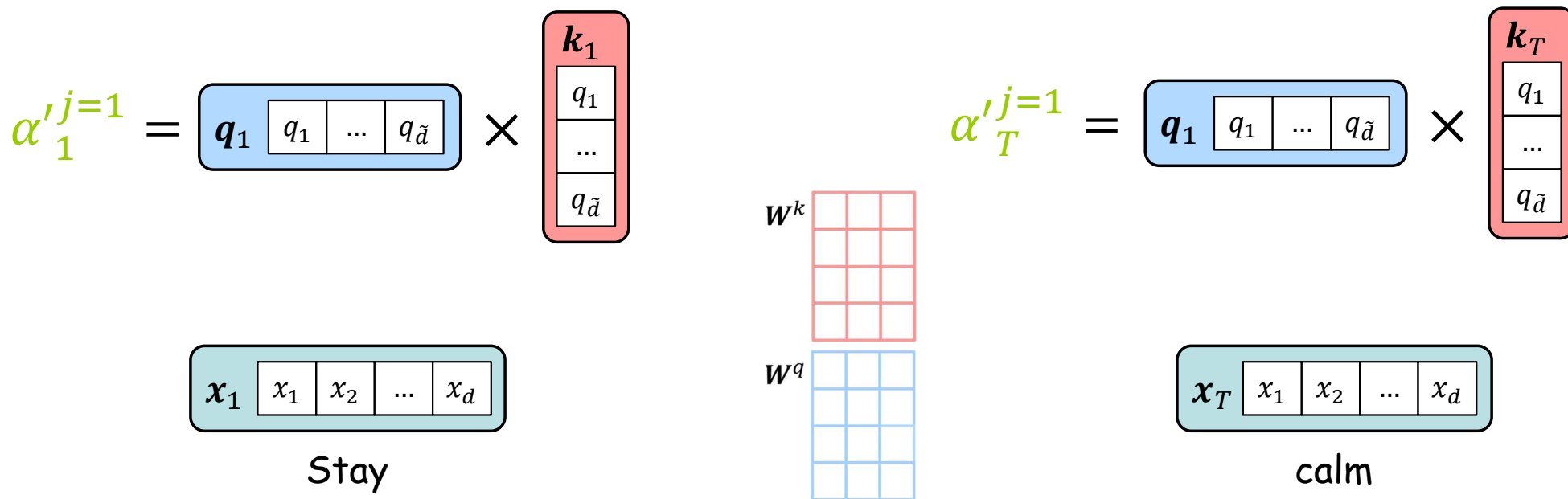


Self-Attention in Detail

Say we focus on token $j = 1$

■ Compute **raw scores** α'^j_t as dot product of query and key vectors $q_j \cdot k_t^\top \forall t = 1, \dots, T$

- Recall that the dot product is a measure of similarity
- High values at same positions produce high output at this position and vice versa



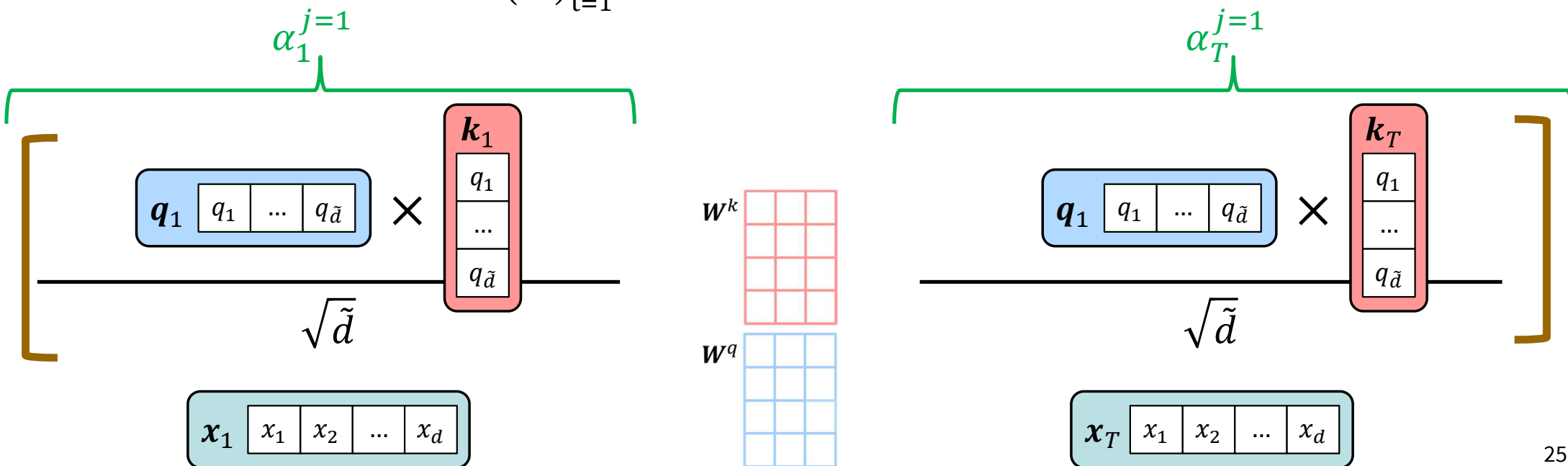
Scaled Dot Product Attention

Say we focus on token $j = 1$

■ Normalize **raw attention scores** using **softmax** to ensure they sum to unity

□ $\alpha_t^j = \exp\left(\frac{q_j \times k_t^T}{\sqrt{\tilde{d}}}\right) / \sum_t \exp\left(\frac{q_j \times k_t^T}{\sqrt{\tilde{d}}}\right)$

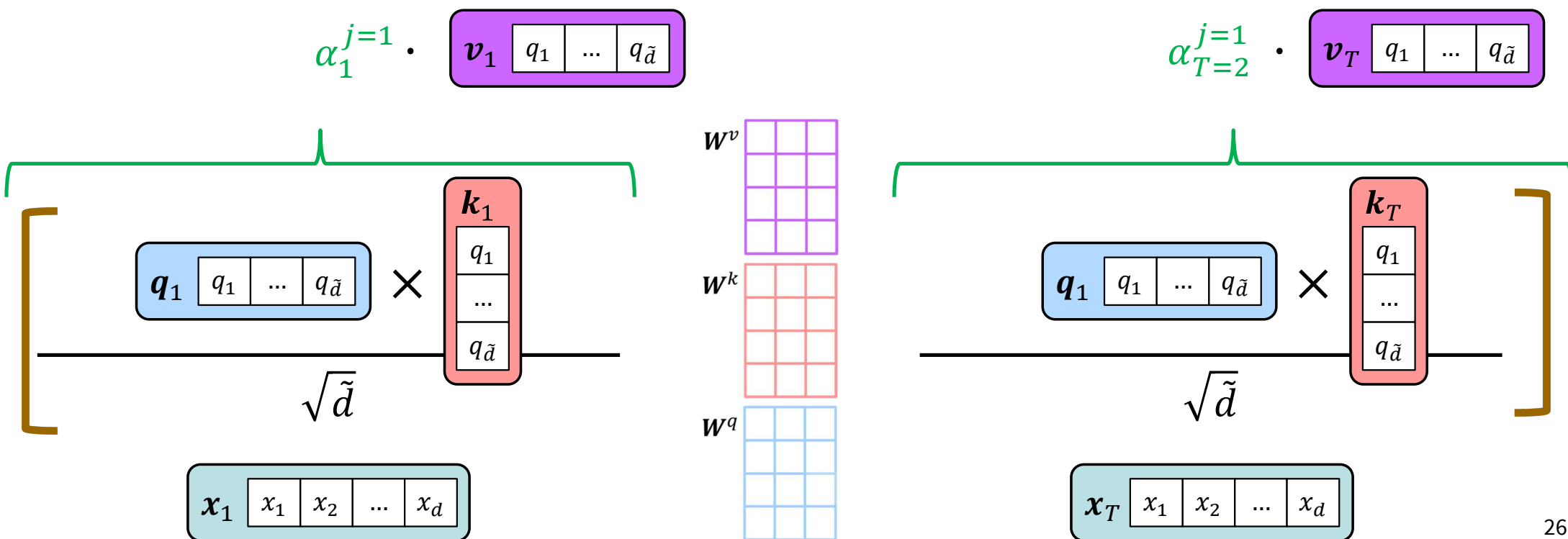
□ Normalized scores $\alpha^j = \left(\alpha_t^j\right)_{t=1}^T$ determine how much each token is expressed at position j



Scaled Dot Product Attention

Say we focus on token $j = 1$

- Multiply **attention scores** with **value vectors** $\alpha_t^j v_t$

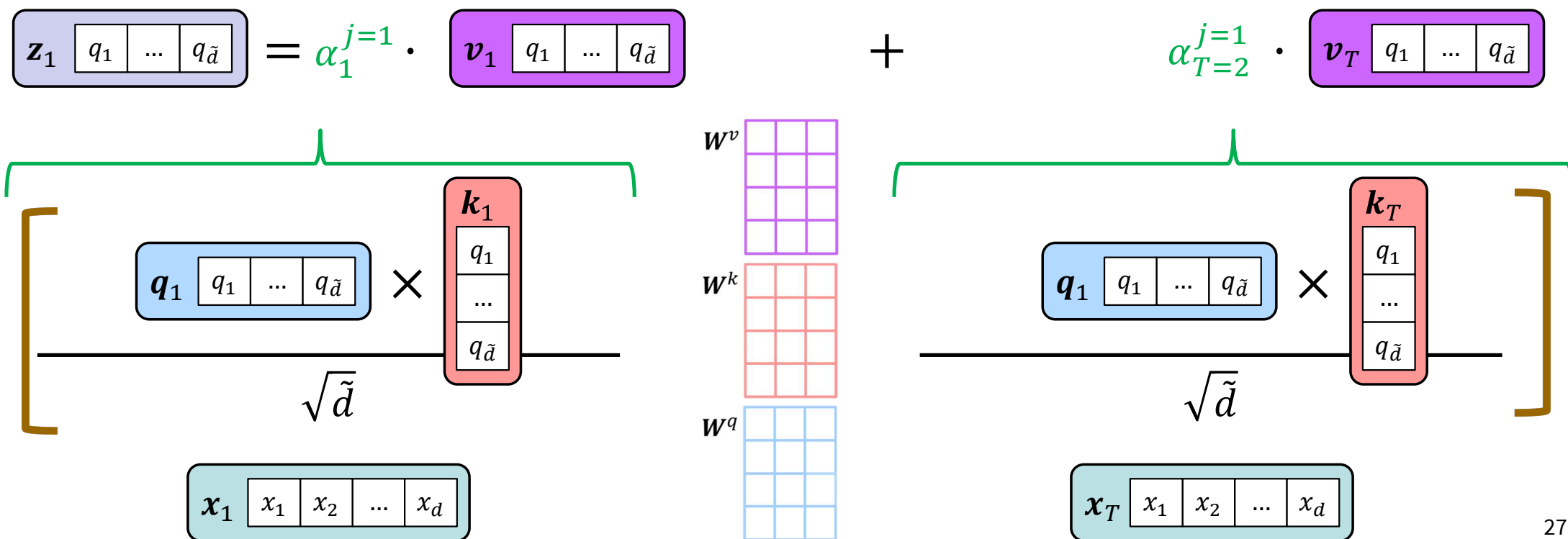


Scaled Dot Product Attention

Say we focus on token $j = 1$

■ Multiply **attention scores** with **value vectors** $\alpha_t^j v_t$

■ Compute attention output for this position, z_j , as weighted sum $z_j = \sum_{t=1}^T \alpha_t^j v_t$



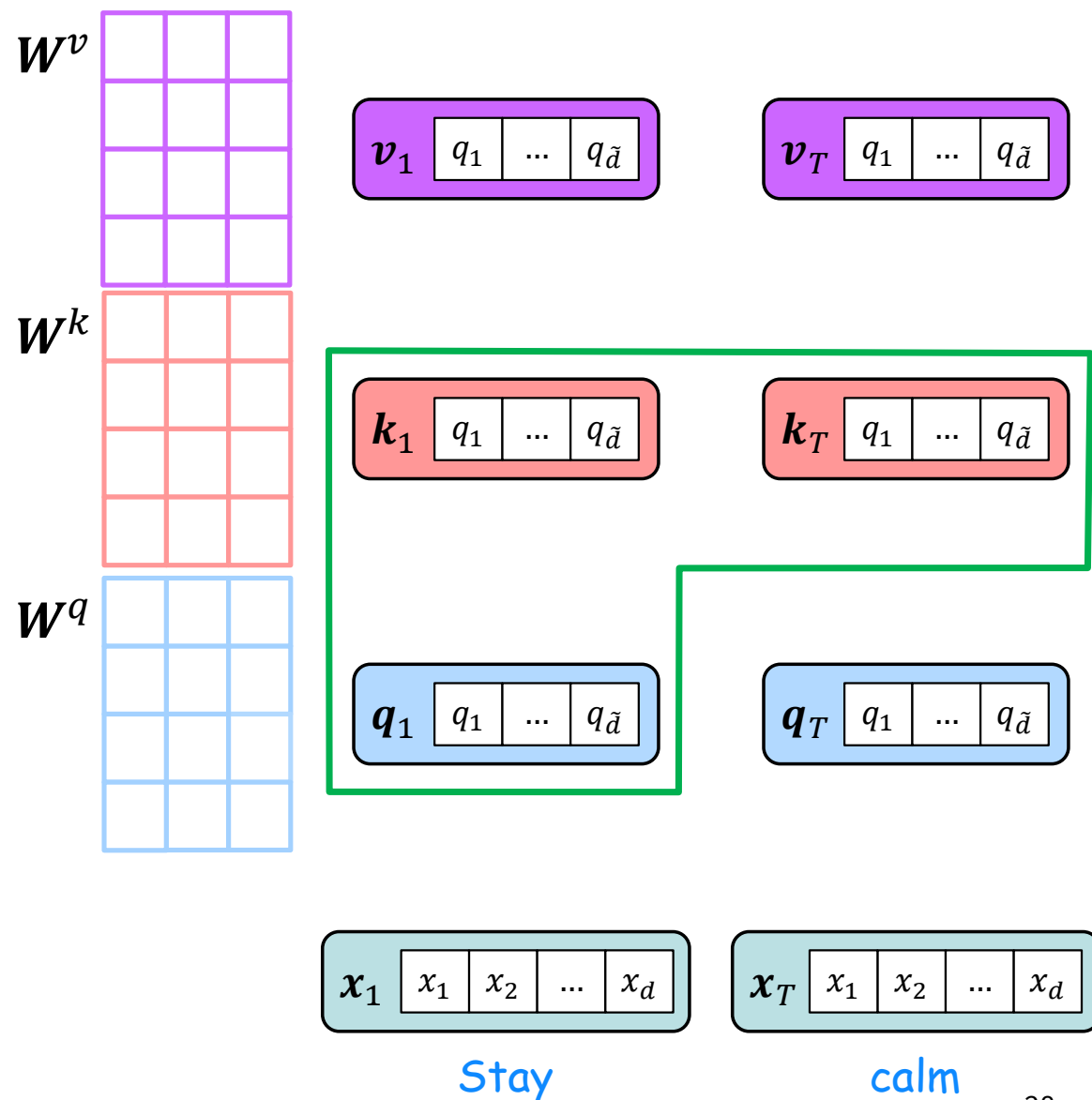
Scaled Dot Product Attention

Summary for tokens $j = 1$

■ For $j = 1$ we use q_1 to compute scores

$$\mathbf{z}_1 \begin{bmatrix} q_1 & \dots & q_{\tilde{d}} \end{bmatrix} = \alpha_1^{j=1} \times \mathbf{v}_1 \begin{bmatrix} q_1 & \dots & q_{\tilde{d}} \end{bmatrix} + \alpha_2^{j=1} \times \mathbf{v}_T \begin{bmatrix} q_1 & \dots & q_{\tilde{d}} \end{bmatrix}$$

$$\alpha^{j=1} = \text{softmax} \left(\frac{\mathbf{q}_1 \mathbf{k}_1}{\sqrt{\tilde{d}}}, \dots, \frac{\mathbf{q}_1 \mathbf{k}_T}{\sqrt{\tilde{d}}} \right)$$



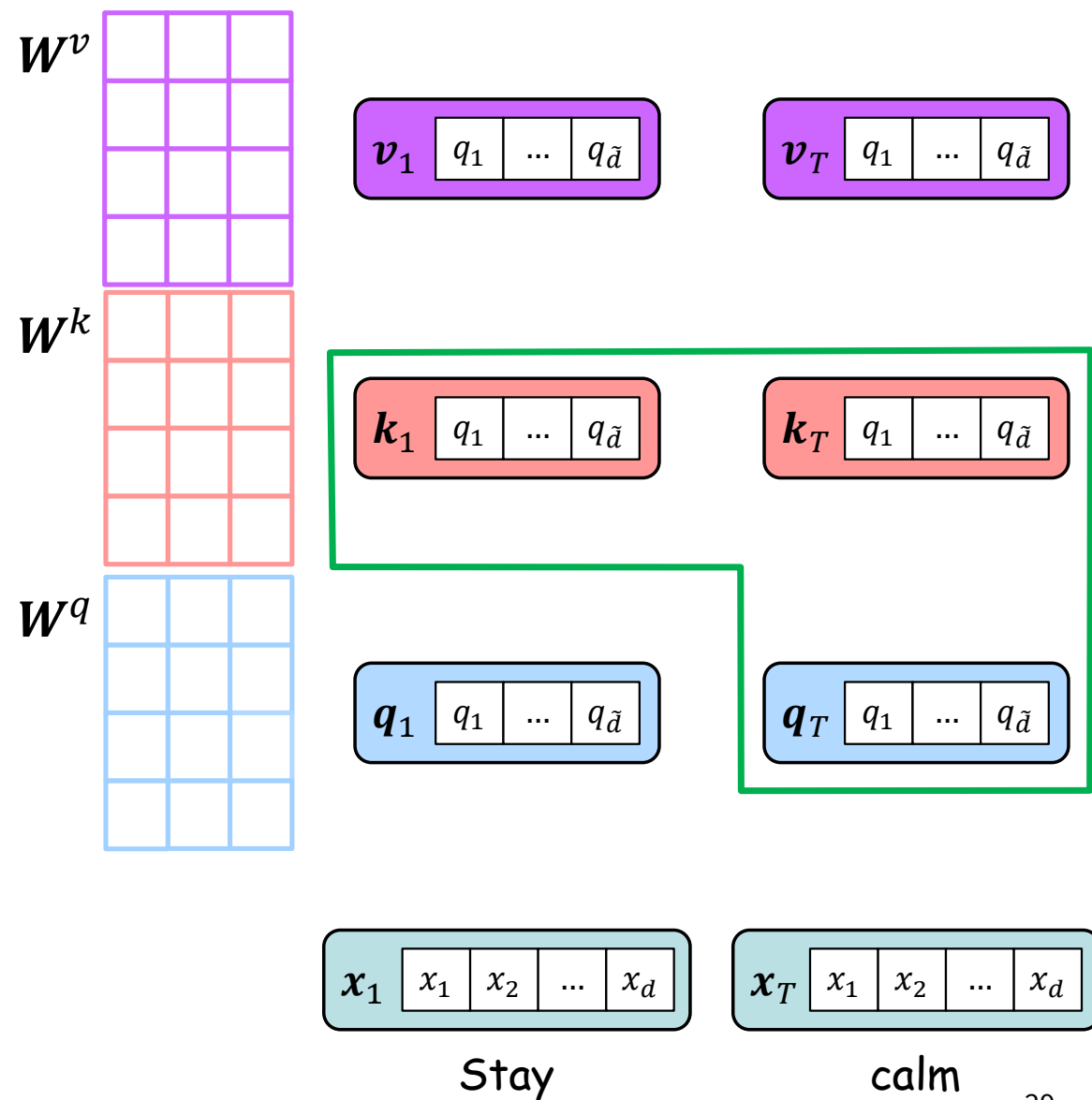
Scaled Dot Product Attention

Proceed alike for other tokens j

■ Now let $j = 2 = T$

$$\mathbf{z}_2 \begin{bmatrix} q_1 & \dots & q_{\tilde{d}} \end{bmatrix} = \alpha_1^{j=2} \times \mathbf{v}_1 \begin{bmatrix} q_1 & \dots & q_{\tilde{d}} \end{bmatrix} + \alpha_2^{j=2} \times \mathbf{v}_T \begin{bmatrix} q_1 & \dots & q_{\tilde{d}} \end{bmatrix}$$

$$\alpha^{j=2} = \text{softmax} \left(\frac{\mathbf{q}_2 \mathbf{k}_1}{\sqrt{\tilde{d}}}, \dots, \frac{\mathbf{q}_2 \mathbf{k}_T}{\sqrt{\tilde{d}}} \right)$$

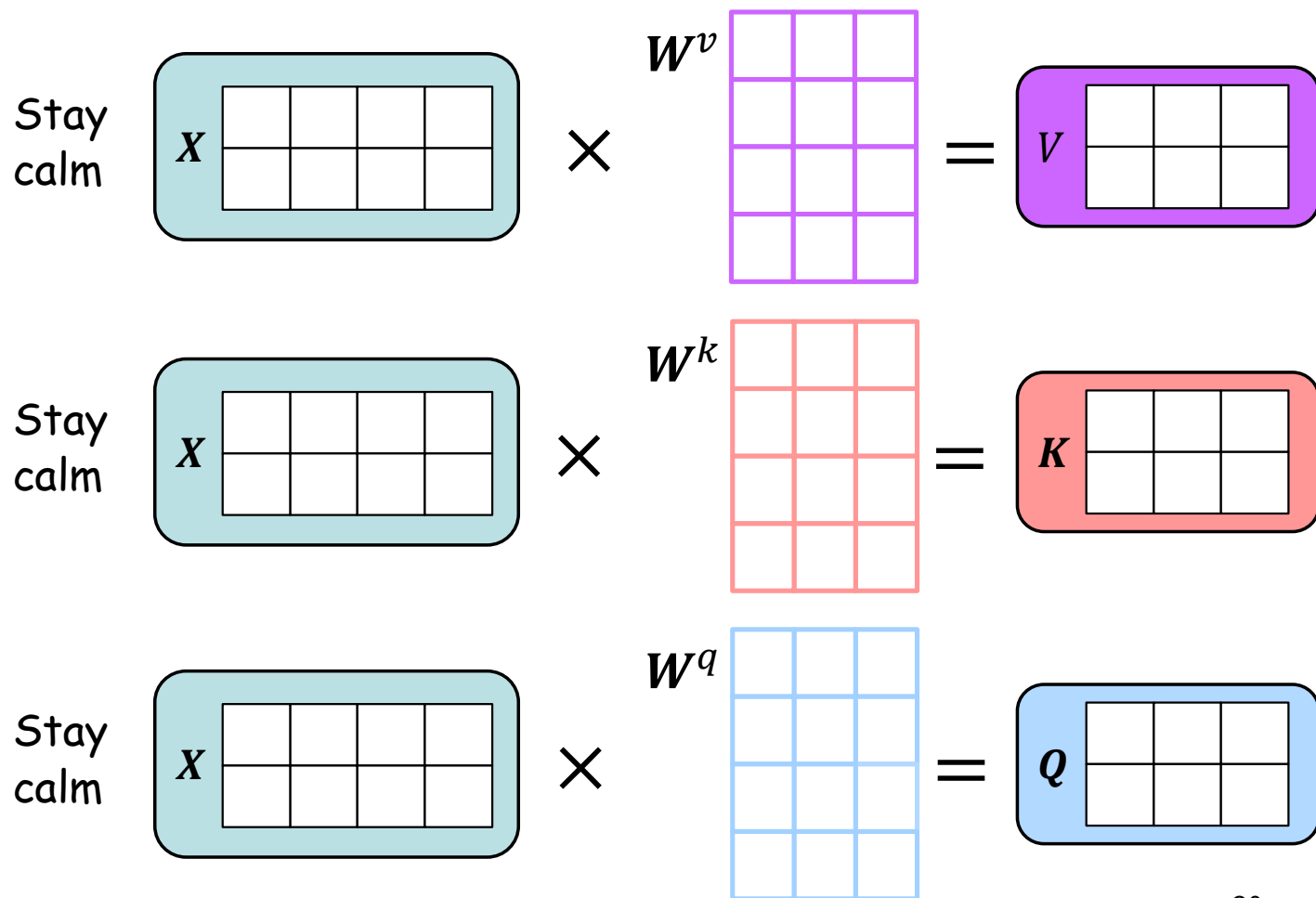


Vectorize Calculations for Efficiency

In practice, we compute all attention outputs in parallel

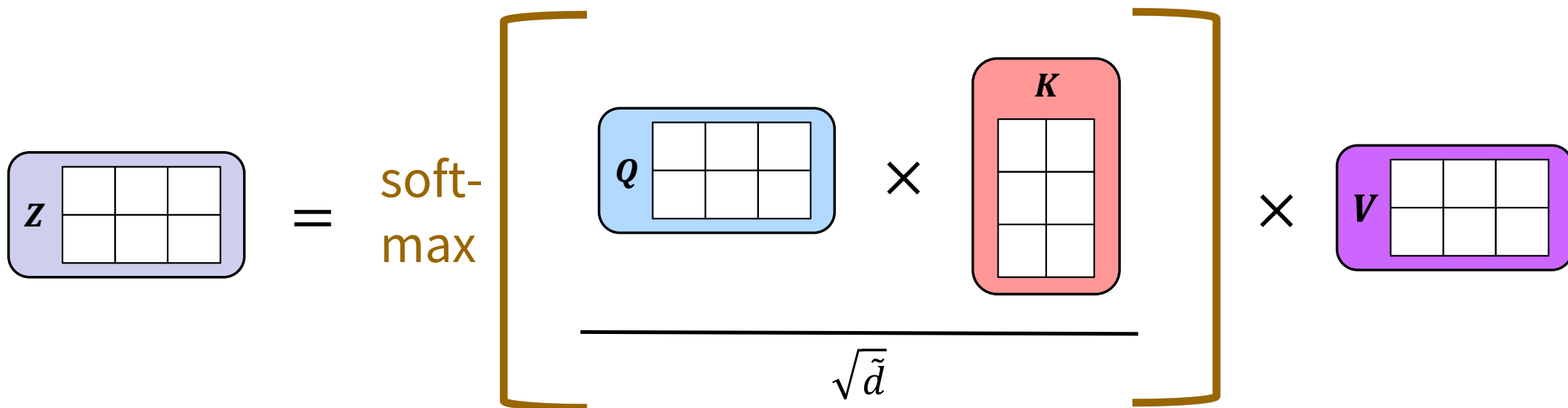
- Input matrix of stacked token embeddings
- Compute **query**, **key**, and **value** matrices for entire input sequence

- No recurrence
- Highly efficient



Vectorize Calculations for Efficiency

Scaled dot product attention in matrix format



The diagram illustrates the Scaled Dot Product Attention mechanism in matrix format. On the left, a purple rounded rectangle labeled Z contains a 2x3 grid. This is followed by an equals sign and the text "soft-max" in brown. To the right of "soft-max" is a large brown bracket. Inside the bracket, a blue rounded rectangle labeled Q (2x3 grid) is multiplied by a red rounded rectangle labeled K (3x3 grid). A horizontal line is drawn below the multiplication, with the label $\sqrt{\tilde{d}}$ centered underneath it. To the right of the bracket is a multiplication symbol $\times$ followed by a purple rounded rectangle labeled V (2x3 grid).

$$Z = \text{soft-max} \left[\frac{Q \times K}{\sqrt{\tilde{d}}} \right] \times V$$

Multi-Head Attention

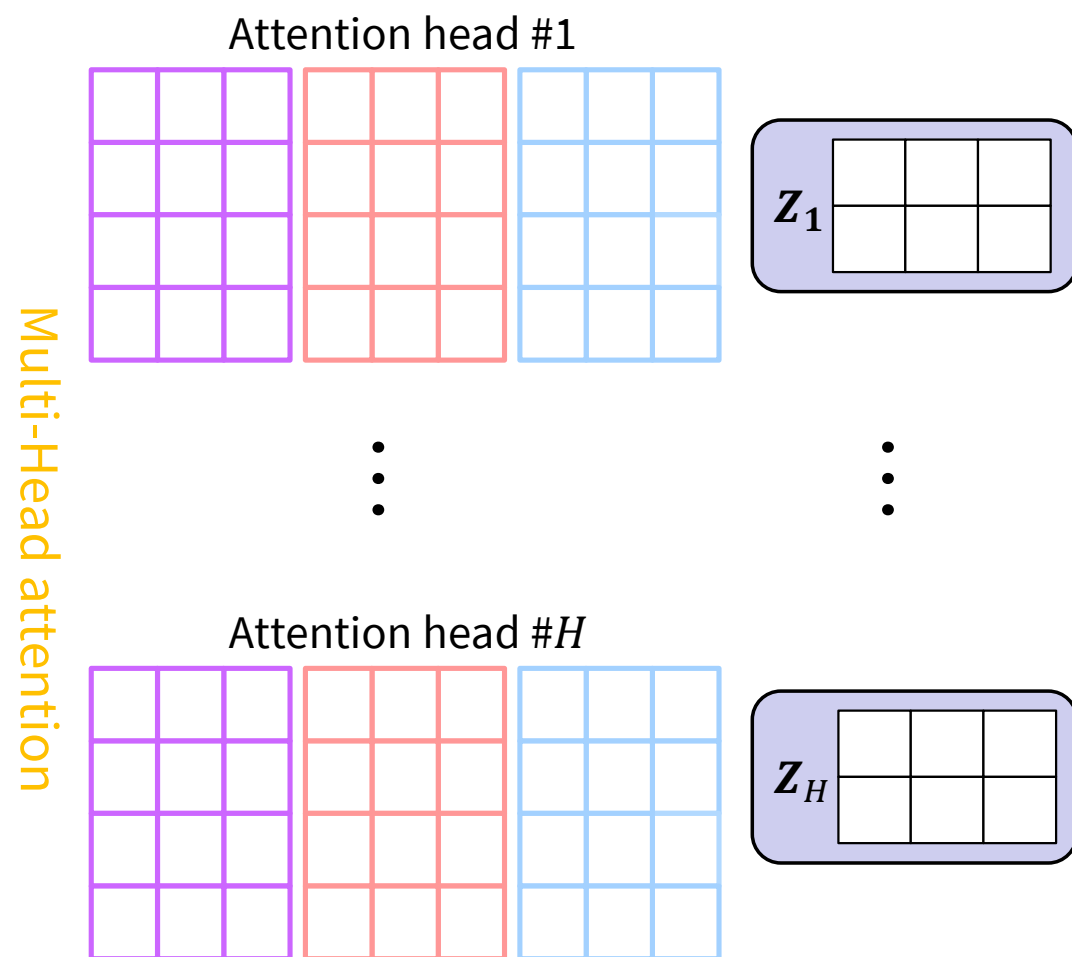
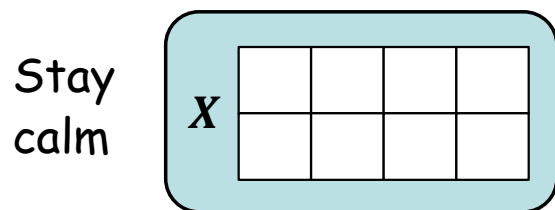
Repeat attention calculations raises representational capacity

■ Scaled dot product attention

- Transformation of one input sequence, X into one output sequence, Z
- The computational units performing this operations are called an *attention head*

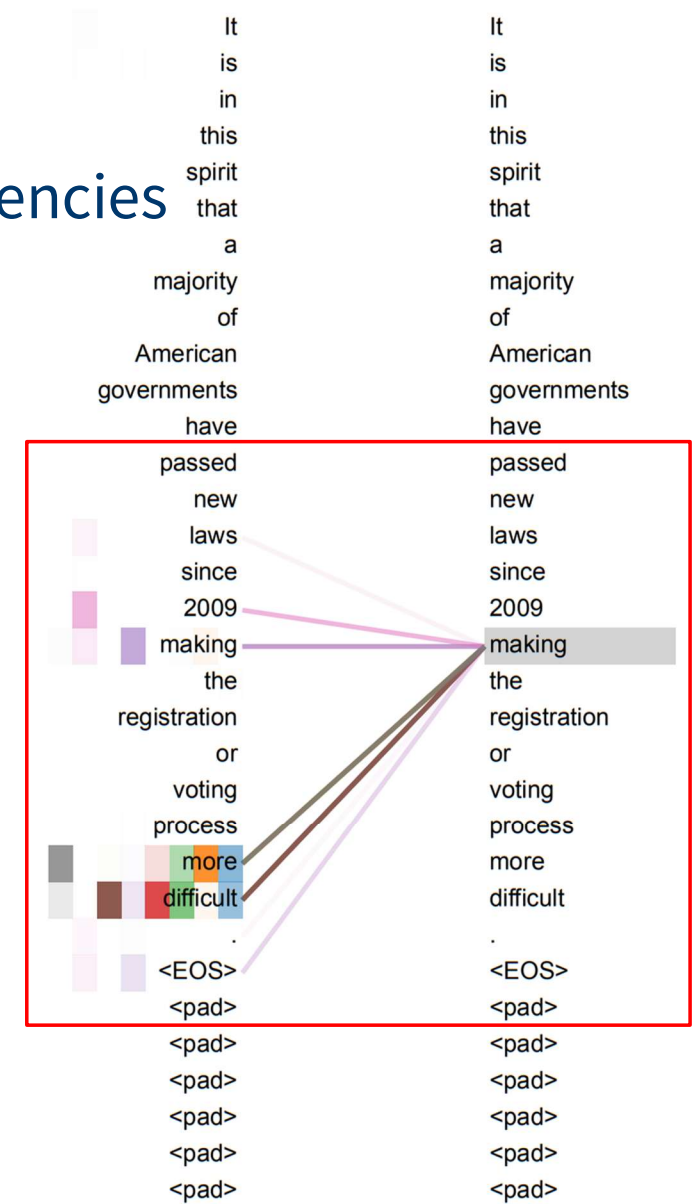
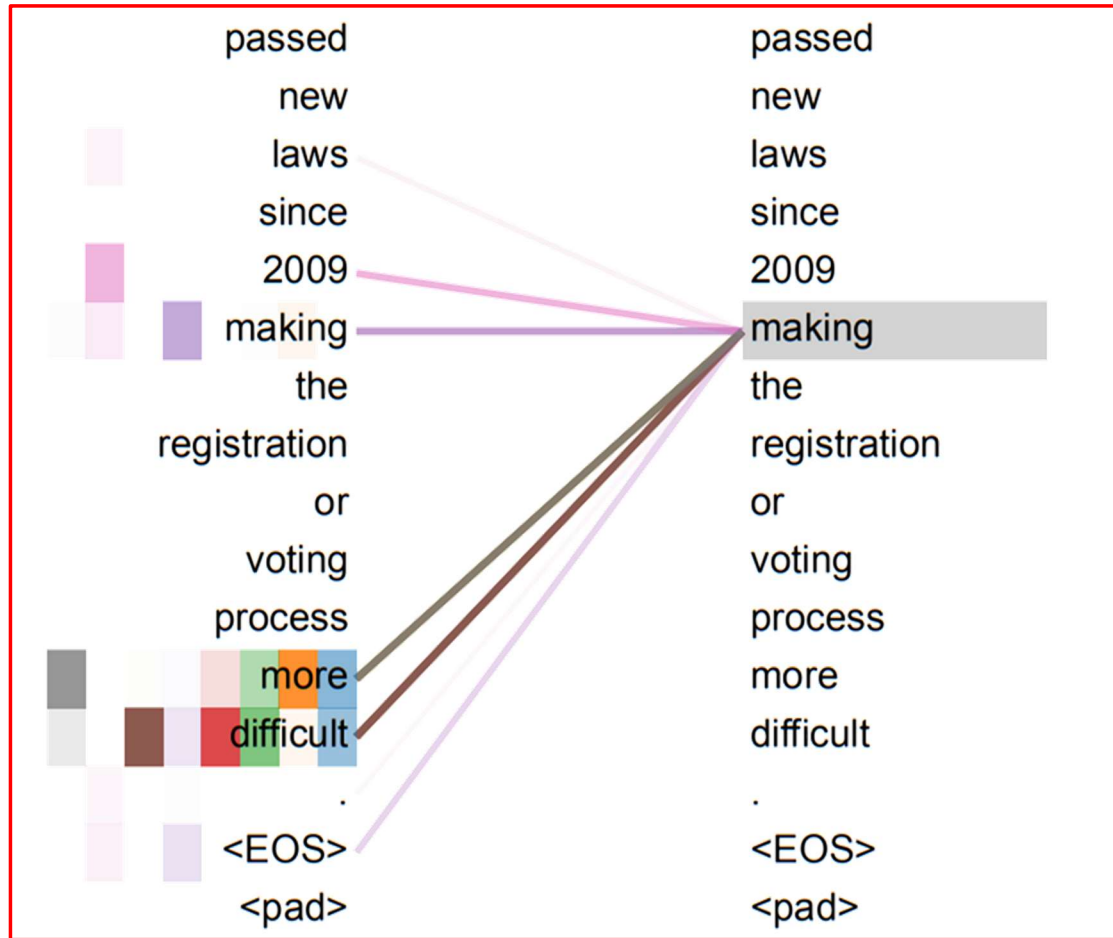
■ The architecture of Vaswani et al. (2017) comprises 8 *attention heads*

- More heads $\rightarrow$ more weights $\rightarrow$ more flexibility
- Each head can focus on different dependencies
- Attention layer can map input to multiple sub-spaces



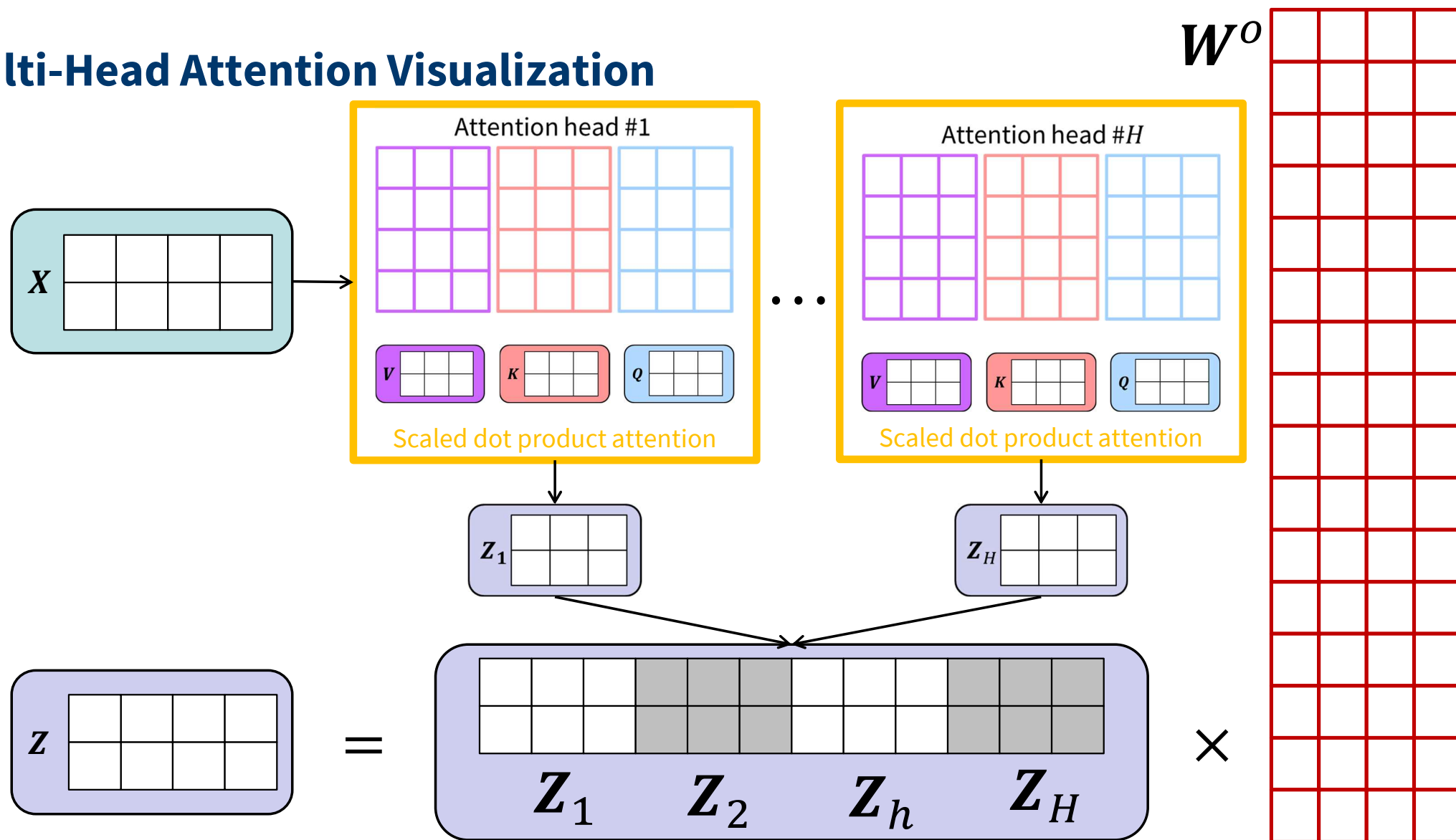
Multi-Head Attention

Different attention heads capture different dependencies



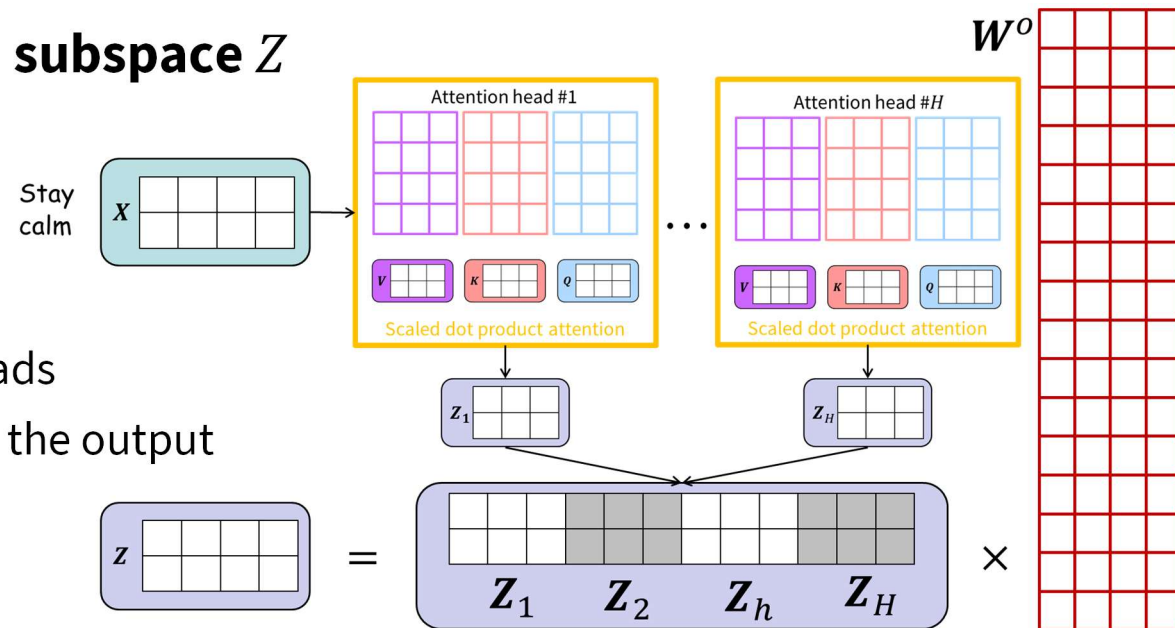
Multi-Head Attention Visualization

Stay calm



Multi-Head Attention Summary

- Each attention head maps input X to subspace Z
- Stack subspaces
- Multiply with properly shaped **output weight matrix** $W^O \in \mathbb{R}^{\tilde{d}H \times d}$
 - Aggregate outputs of different attention heads
 - Gives full control over the dimensionality of the output
- **Attention layer does not change the dimensionality of the input**
 - Useful to devise a deep architecture with many stacked layers
 - All calculation run in parallel w/o recurrence



Formal Summary of Multi-Head Attention

■ Project X to queries, keys, and values for each attention head

$$\square Q_{(h)} = XW_{(h)}^Q \in \mathbb{R}^{T \times \tilde{d}}$$

$$\square K_{(h)} = XW_{(h)}^K \in \mathbb{R}^{T \times \tilde{d}}$$

$$\square V_{(h)} = XW_{(h)}^V \in \mathbb{R}^{T \times \tilde{d}}$$

■ Compute attention output per head

$$\square Z_{(h)} = \text{Attention}_{(h)}(Q_{(h)}, K_{(h)}, V_{(h)}) \in \mathbb{R}^{T \times \tilde{d}}$$

$$\square \text{Attention}_{(h)}(Q_{(h)}, K_{(h)}, V_{(h)}) = \text{softmax}\left(\frac{Q_{(h)}K_{(h)}^T}{\sqrt{\tilde{d}}}\right)V_{(h)}$$

■ Aggregate outputs of different heads

$$\square Z_{\text{concat}} = (Z_{(1)}, Z_{(2)}, \dots, Z_{(H)}) \in \mathbb{R}^{T \times d}$$

$$\square Z = Z_{\text{concat}}W^O \in \mathbb{R}^{T \times d}$$

$\square T$ sequence length

$\square d$ embedding of dimension

$\square \tilde{d} = d/H$ dimension of query, key, and value

$\square H$ number of attention heads

$\square X \in \mathbb{R}^{T \times d}$ input token representation

$\square Q_{(h)}, K_{(h)}, V_{(h)} \in \mathbb{R}^{T \times \tilde{d}}$ query, key, and value matrices in head h

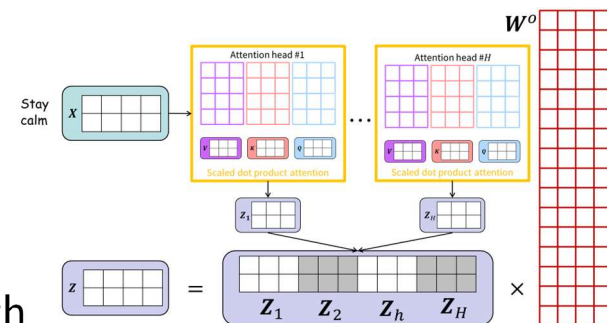
$\square W_{(h)}^Q, W_{(h)}^K, W_{(h)}^V \in \mathbb{R}^{d \times \tilde{d}}$ projection matrices for Q, K, V in head h

$\square Z_{(h)} \in \mathbb{R}^{T \times \tilde{d}}$ attention output of head h

$\square Z_{\text{concat}} \in \mathbb{R}^{T \times d}$ concatenation of all heads' outputs

$\square W^O \in \mathbb{R}^{d \times d}$ output projection matrix

$\square Z \in \mathbb{R}^{T \times d}$ final output of multi-head self-attention



Transformer Block Summary

Map input sequence to a contextualized output sequence of the same length

■ Multi-head attention layer

- Capture dependencies among input tokens through linear projections
- Designed to sustain dimensionality

■ Normalization step

- Add attention layer output and its input coming from residual connection
- Aim is to stabilize gradients

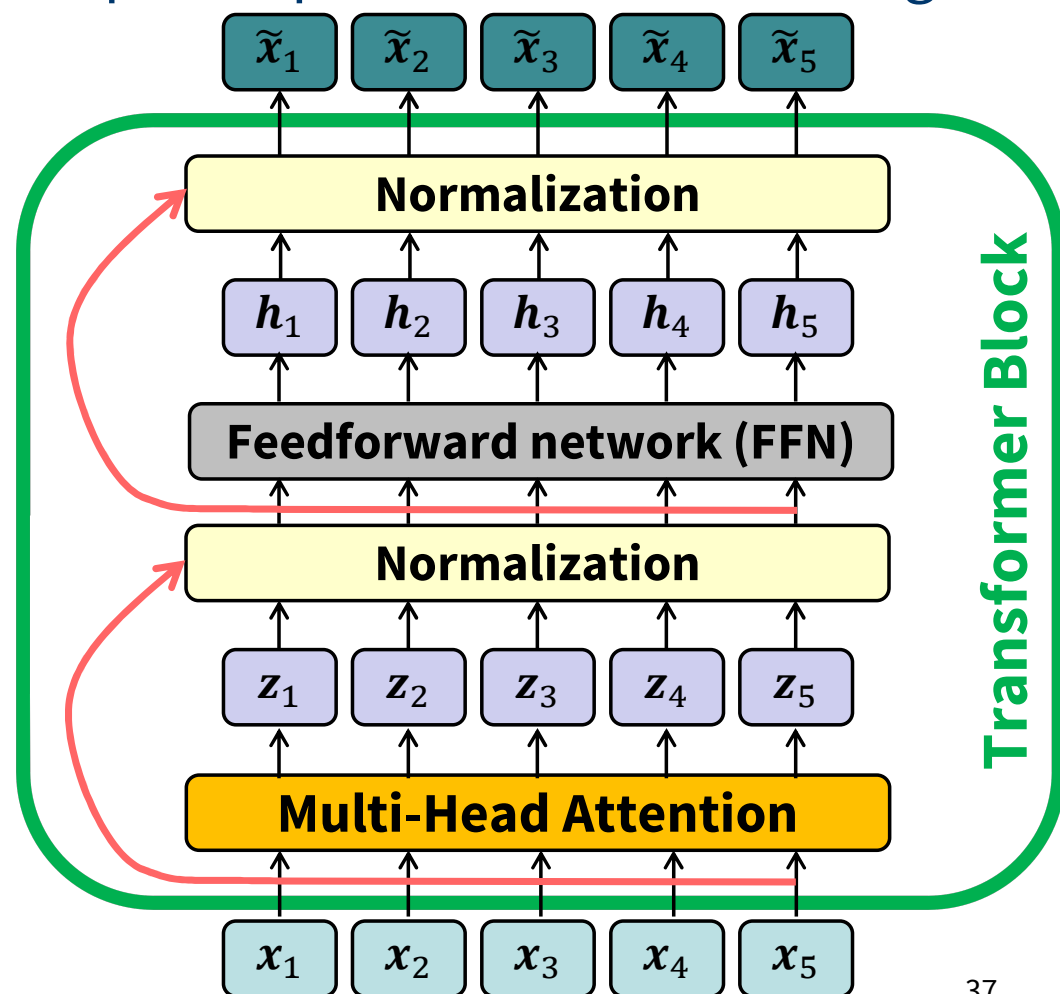
■ Feedforward neural network

- Applied position-wise to input sequence
 - Weight sharing across tokens enforces consistency across positions
 - Enables parallel token processing
- Adds nonlinearity through RELU activation
- Designed to sustain dimensionality

■ Another normalization step

■ Output = context-aware representation of input

- Length of the input/output sequence is a design parameter
- Closely related to the *context window* of an LLM



Masked Multi-Head Attention

Decoder attention must not look ahead during decoding

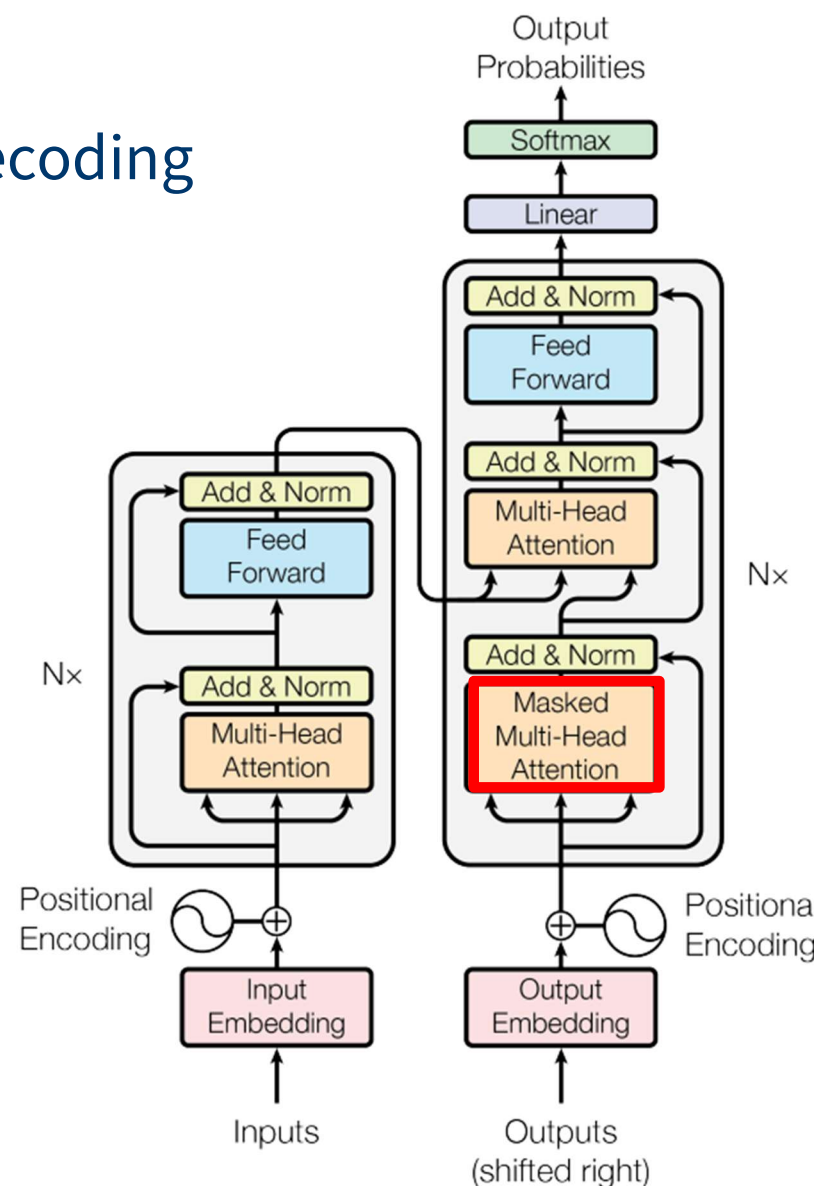
■ At position j , we can only attend to positions $i \leq j$

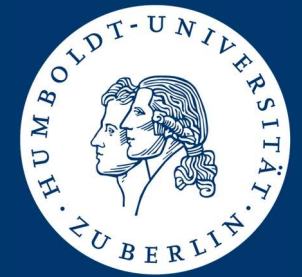
- Recall machine translation example
- How a sentence continues is unknown
- Achieved through masking

■ Masked multi-head attention (MMHA)

$$\text{MMHA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{\tilde{d}}} + \mathbf{M}\right)\mathbf{V}$$

- with $\mathbf{M}$ denoting a triangular matrix
- $M_{ji} = \begin{cases} 0 & \text{if } j \geq i \\ -\infty & \text{if } j < i \end{cases}$ where j is the current position
- Softmax assigns zero weight to positions $> j$
- Also *causal* attention (i.e., “past causes future”)





1010
1010

Positional Encoding

Transformer Block has no Inherent Sense of Token Order

All tokens are processed simultaneously and independently

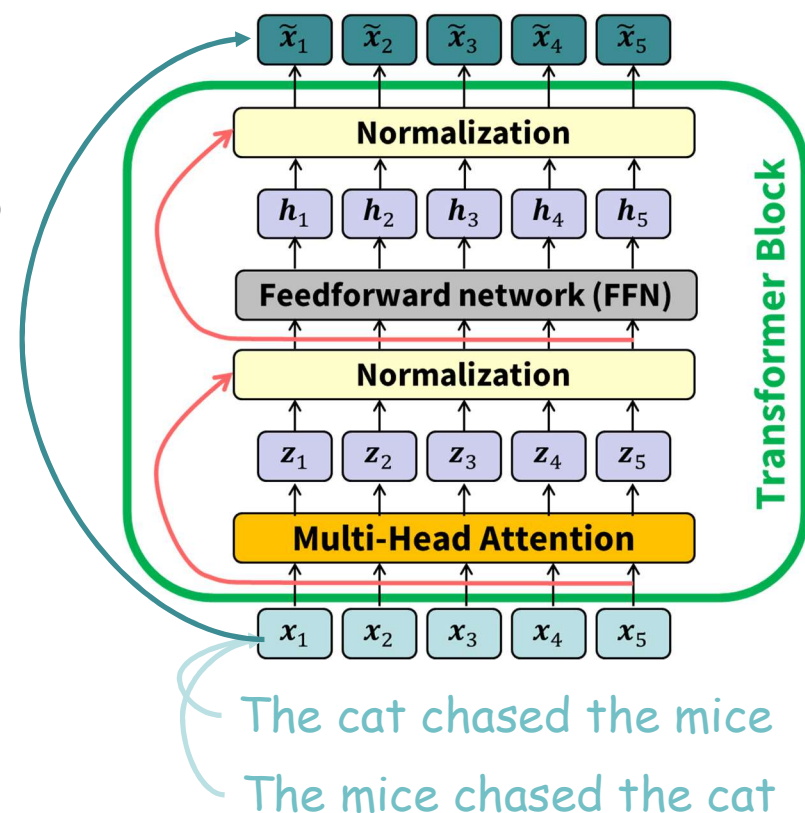
- **Self-attention is permutation-invariant**

- Treat input as a “bag of embeddings”
- Each token is compared to every other token of the sequence to generate attention scores

- **Transformer block cannot distinguish sequences of same content**

- **Order awareness is crucial for capturing semantic and syntactic relationships**

- **Need auxiliary mechanism to capture not just the presence of words, but their positions relative to each other**



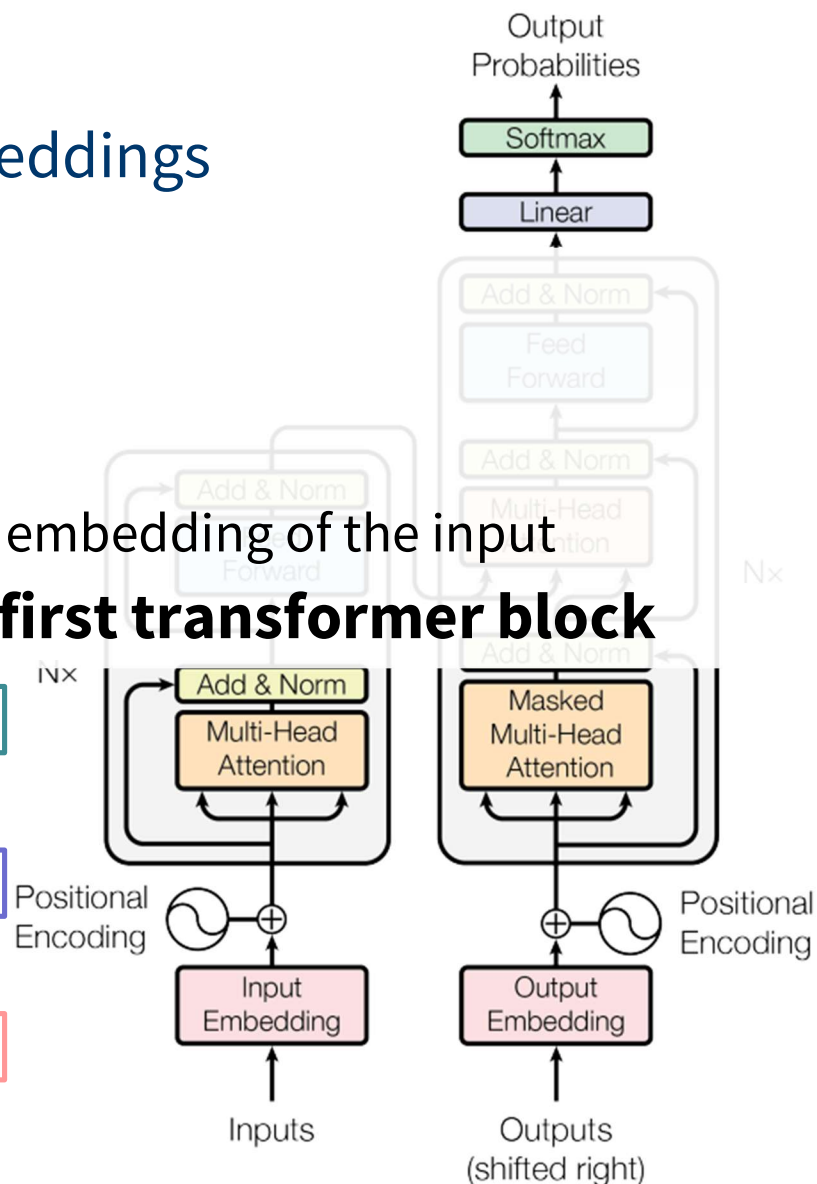
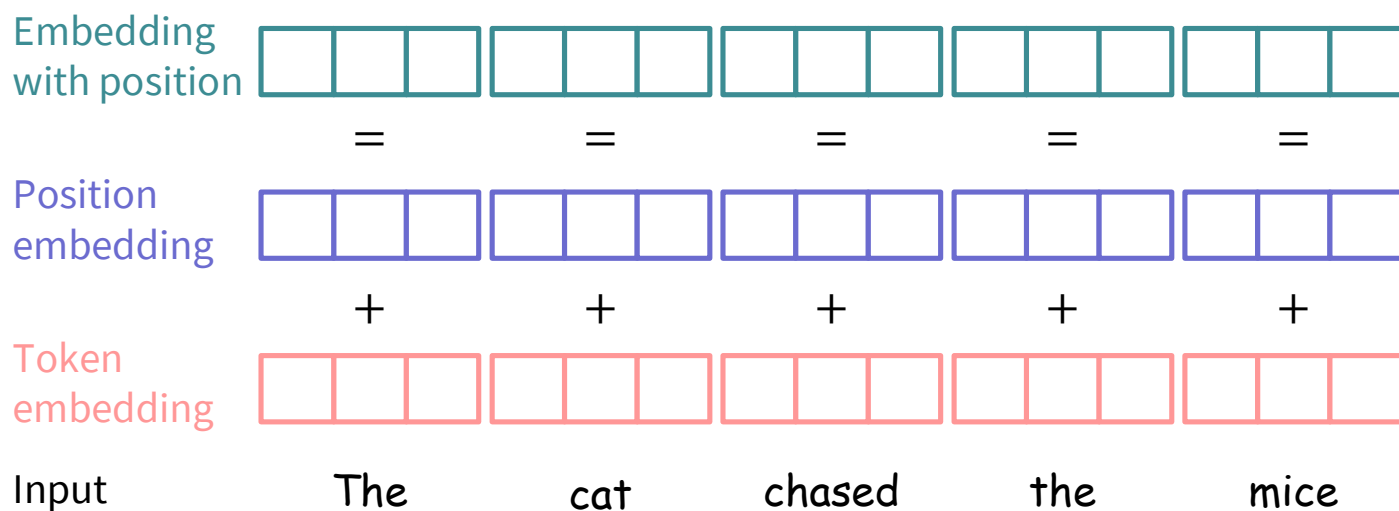
Positional Encoding (PE)

Inject positional information into the token embeddings

■ Refine the embedding of model inputs

- Raw embedding represents the content of a token
- Also *embed* the position of a token
- Add the two embeddings to obtain a position-aware embedding of the input

■ Add PE before processing input sequence in first transformer block



Types of Positional Encodings

Main types include fixed and trainable positional embeddings

■ Vaswani et al. (2017) introduced fixed positional encodings

- Predefined sine & cosine functions of different frequencies
- Generalize to sequence lengths not seen during training
- Less flexible because PE is fixed for all sequences → risk to underfit in position-sensitive tasks

■ Many modern LLMs employ trainable PEs

- Learn position vectors during model training
 - PE are treated just like any other parameter
 - Add a matrix $\mathbf{P} \in \mathbb{R}^{[\text{max sequence length}] \times [\text{embedding dimension}]}$ of randomly initialized weights to the input
 - Model training turns these weights into meaningful positional signals
- Potentially better performance on in-distribution data due to adaptiveness
- No extrapolation to longer sequences (i.e., beyond predefined maximal sequence length)

Fixed Positional Encoding Based on Vaswani et al. (2017)

■ Plug each token position $t = 0, 1, 2, \dots$ into a smooth sinusoidal function

□ $PE_{(t, 2i)} = \sin\left(t/n^{2i/d}\right)$; $PE_{(t, 2i+1)} = \cos(t/n^{2i/d})$

□ where i indexes the PE dimension, $i = 0, \dots, (d/2 - 1)$, d is the embedding dimension (e.g., 512 in the paper), and n is a user-defined scaler; set to 10,000 in Vaswani et al. (2017)

0	$\sin(0/n^{0/d})$	0	$\sin(1/n^{0/d})$	0	$\sin(2/n^{0/d})$	0	$\sin(3/n^{0/d})$	0	$\sin(4/n^{0/d})$
1	$\cos(0/n^{1/d})$	1	$\cos(1/n^{1/d})$	1	$\cos(2/n^{1/d})$	1	$\cos(3/n^{1/d})$	1	$\cos(4/n^{1/d})$
2	$\sin(0/n^{2/d})$	2	$\sin(1/n^{2/d})$	2	$\sin(2/n^{2/d})$	2	$\sin(3/n^{2/d})$	2	$\sin(4/n^{2/d})$
3	$\cos(0/n^{3/d})$	3	$\cos(1/n^{3/d})$	3	$\cos(2/n^{3/d})$	3	$\cos(3/n^{3/d})$	3	$\cos(4/n^{3/d})$
...	...	...	...	...	...	...	...	...	...
511	$\cos(0/n^{510/d})$	511	$\cos(1/n^{510/d})$	511	$\cos(2/n^{510/d})$	511	$\cos(3/n^{510/d})$	511	$\cos(4/n^{510/d})$

Position
embedding

Token
position t
Input

0

The

1

cat

2

chased

3

the

4

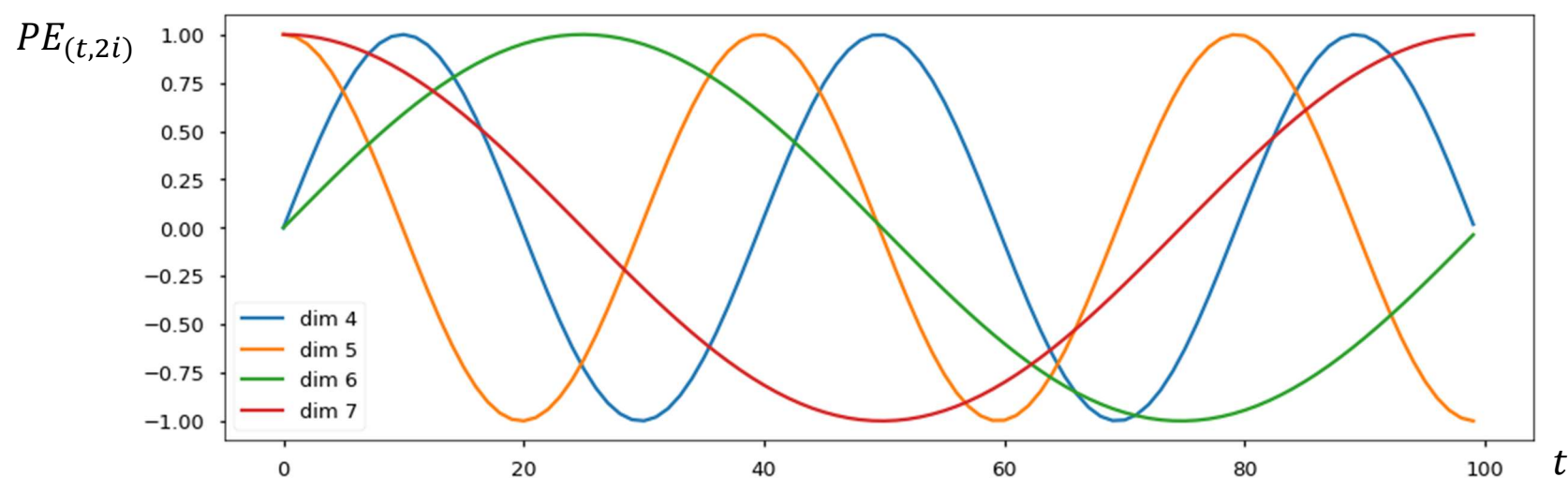
mice

Fixed Positional Encoding Based on Vaswani et al. (2017)

■ Plug each token position $t = 0, 1, 2, \dots$ into a smooth sinusoidal function

□ $PE_{(t,2i)} = \sin\left(t/n^{\frac{2i}{d}}\right)$; $PE_{(t,2i+1)} = \cos(t/n^{2i/d})$

□ where i indexes the PE dimension, $i = 0, \dots, (d/2 - 1)$, d is the embedding dimension (e.g., 512 in the paper), and n is a user-defined scaler; set to 10,000 in Vaswani et al. (2017)



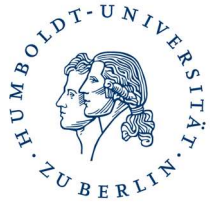


Some Popular Transformers

GPT, BERT, and beyond

Transformers Everywhere

Transformer architecture became de facto standard in many fields

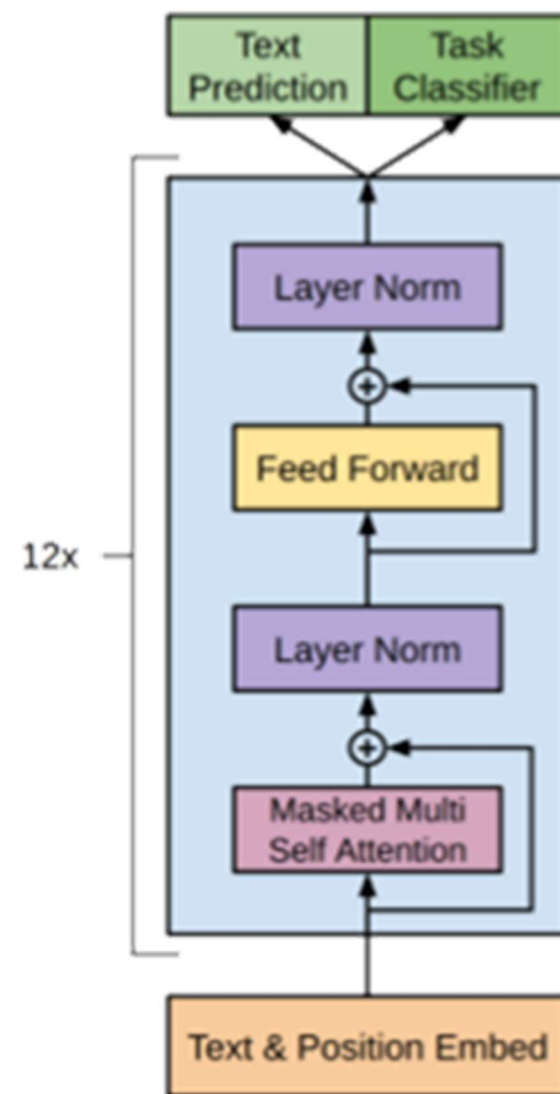


- **Following the success of the transformer in NLP, transformer-based approaches for computer vision, time series forecasting, and others emerged and increasingly replaced alternatives**
- **Multi-modal models also rely on the transformer architecture**
- **Readying a transformer for non-text data often involves adjusting the attention mechanism as its key component**
- **Further, and even within NLP, the functionality and performance of a transformer network depends on how it is trained**
 - Different tasks benefit from specialized training protocols
 - Speaking about training, how actually did Vaswani et al. (2017) train their transformer???
 - Turns out that training a model for machine translation is more demanding compared to the training in some famous models that followed the transformer paper
 - Let's look at these first

Generative Pre-Training Transformer (GPT)

Using Transformers for Language Modeling

- **Discards transformer encoder part**
- **Stacked decoder blocks (12x)**
 - Masked multi-headed self-attention
 - Pointwise feed-forward layer
- **Pre-trained for language modeling**
 - Autoregressive token processing
 - Model uses it's previously generated output to predict the next token
- **Supervised finetuning for downstream tasks**
 - Transfer learning was a key focus in the GPT paper
 - Authors demonstrated GPT's capabilities in zero-shot learning
- **More enhancements in GPT-2, GPT-3, etc.**



Generative Pre-Training Transformer (GPT)

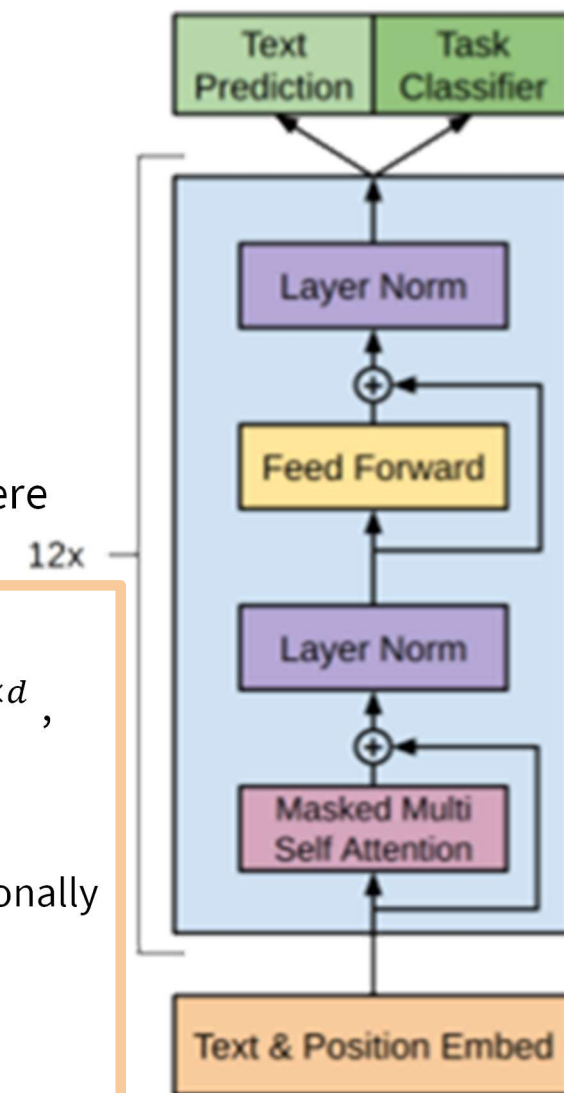
Formal summary of the architecture and pre-training

■ Let

- $\mathcal{V}$ denote a fixed vocabulary of tokens (e.g., subwords or BPE tokens)
- T denote the length of a token sequence
- $\mathbf{y} = (y_1, \dots, y_T)$ denote a sequence of tokens from the training corpus, where each $y_t \in \mathcal{V}$.

■ Embedding and positional encoding

- Map each token y_t to a vector $\mathbf{x}_t^c \in \mathbb{R}^d$ via an embedding matrix $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$, with d denoting the embedding dimension
 - Could be achieved via linear projection $\mathbf{x}_t^c = \mathbf{E}y_t$ if y_t was a one-hot vector
 - Practically, $y_t \in N$ is an index into the vocabulary, so that we avoid a computationally costly dot-product and retrieve the content embedding $\mathbf{x}_t^c$ via indexing
- Compute $\mathbf{x}_t = \mathbf{x}_t^c + \mathbf{p}_t$, where $\mathbf{p}_t \in \mathbb{R}^d$ denotes the PE for position t
- Pass sequence $(\mathbf{x}_1, \dots, \mathbf{x}_T) \in \mathbb{R}^{T \times d}$ into first transformer block



Generative Pre-Training Transformer (GPT)

Formal summary of the architecture and pre-training

■ Transformer block

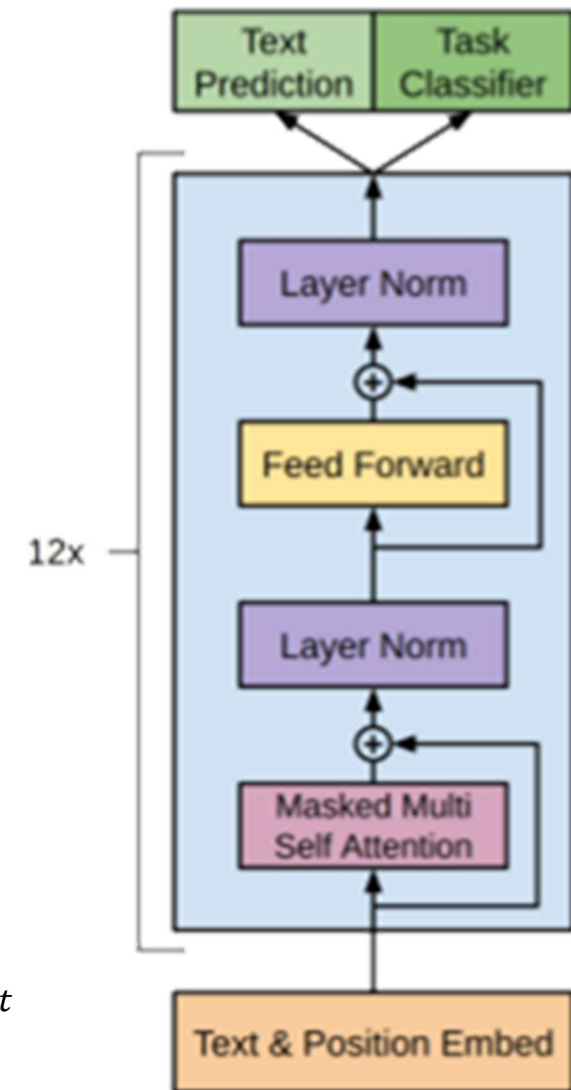
- Applies masked multi headed self-attention (to prevent access to future tokens), position-wise FFN, residual connections, and layer normalization
- Outputs a contextualized embedding $(\tilde{x}_1, \dots, \tilde{x}_T) \in \mathbb{R}^{T \times d}$ as input for next block

■ Last block's output is mapped to a vector of logits over $\mathcal{V}$

- $\mathbf{o}_t = \mathbf{W}_o \tilde{x}_t + \mathbf{b}_o \in \mathbb{R}^{|\mathcal{V}|}$
- with output weight matrix $\mathbf{W}_o \in \mathbb{R}^{|\mathcal{V}| \times d}$ and bias vector $\mathbf{b}_o \in \mathbb{R}^{|\mathcal{V}|}$

■ Logits are converted to probability distribution via softmax

- $P_W(y_t | y_{<t}) = \text{softmax}(\mathbf{o}_{t-1})[y_t]$
- where $[y_t]$ denotes indexing the vector $\text{softmax}(\mathbf{o}_{t-1}) \in \mathbb{R}^{|\mathcal{V}|}$ at position y_t
- and $\mathbf{W}$ represents a matrix of all free parameters of the architecture



Generative Pre-Training Transformer (GPT)

Formal summary of the architecture and pre-training

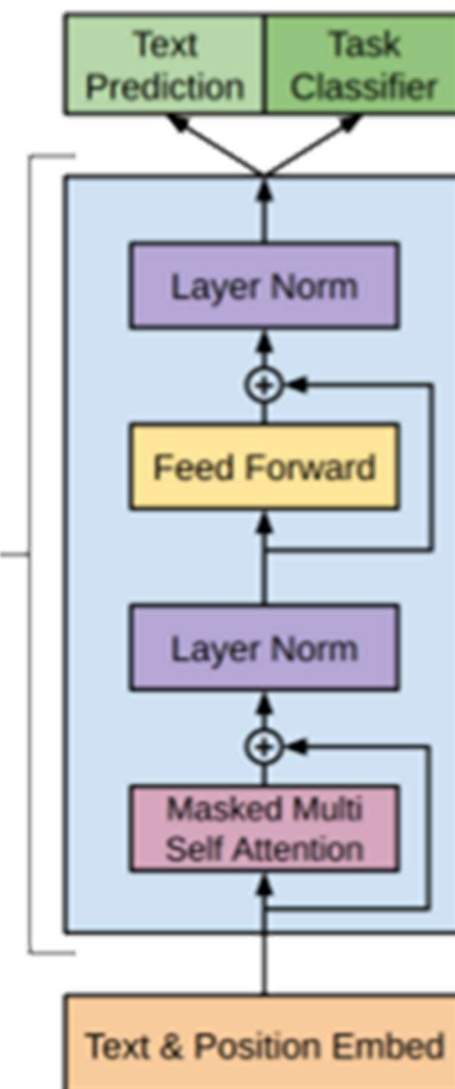
■ GPT pre-training loss

- Maximize log-likelihood of the token sequence under an autoregressive factorization
- Minimize cross-entropy loss between predicted distribution and next token

$$J_{\text{CE}}(\mathbf{W}) = - \sum_{t=1}^T \log(P_{\mathbf{W}}(y_t | y_{<t})) = - \sum_{t=1}^T \log(\text{softmax}(\mathbf{o}_{t-1})[y_t])$$

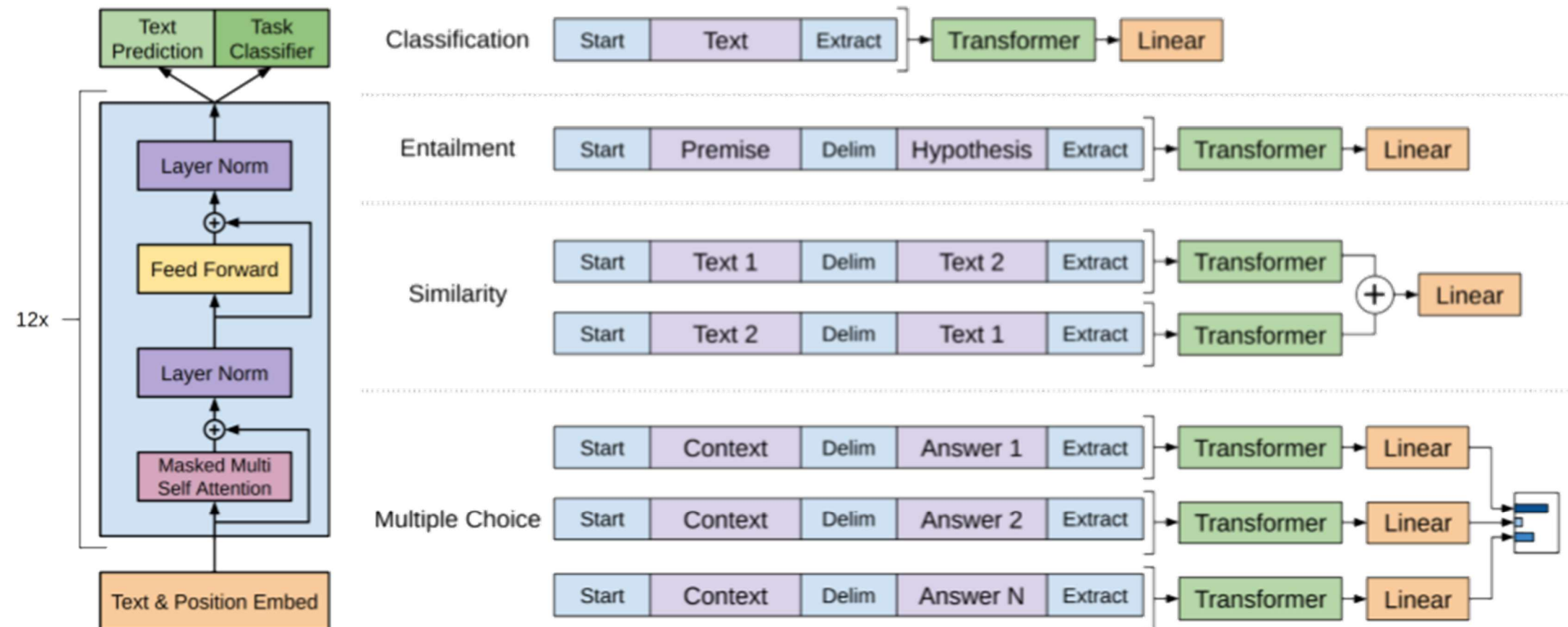
■ Difference between training and inference

- During training, GPT predicts token y_t given the ground-truth prefix $y_1, \dots, y_{t-1}$
 - This strategy is called *teacher forcing*
- At inference, GTP generates tokens one-by-one, feeding each prediction back as input to produce the next token
 - This is called autoregressive decoding or autoregressive generation
 - The process continues until a stopping condition (e.g. end-of-sequence token)



Generative Pre-Training Transformer

New mechanism to use pre-trained model for downstream tasks



Generative Pre-Training Transformer

Base model fine-tuning adopts a multi-task learning perspective

■ Classification example

- Minimize classification loss while also minimizing language model loss

$$J(x) = - \underbrace{\sum_{(y,c) \in \mathcal{D}} \log(P(c|y_1, \dots, y_T))}_{\text{Classification loss}} - \lambda \underbrace{\sum_{t=1}^T \log(P_W(y_t|y_{<t}))}_{\text{Language model loss}}$$

- where c represents a class label (e.g., $c \in \{\text{positive, negative}\}$ in sentiment analysis)

■ Extends to other NLP tasks beyond classification

Bidirectional Encoder Representations from Transformers

Or simply BERT (Devlin et al., 2018)

■ Only the left part of the transformer

- Embed entire input sequence in parallel
- No auto-regressive calculation

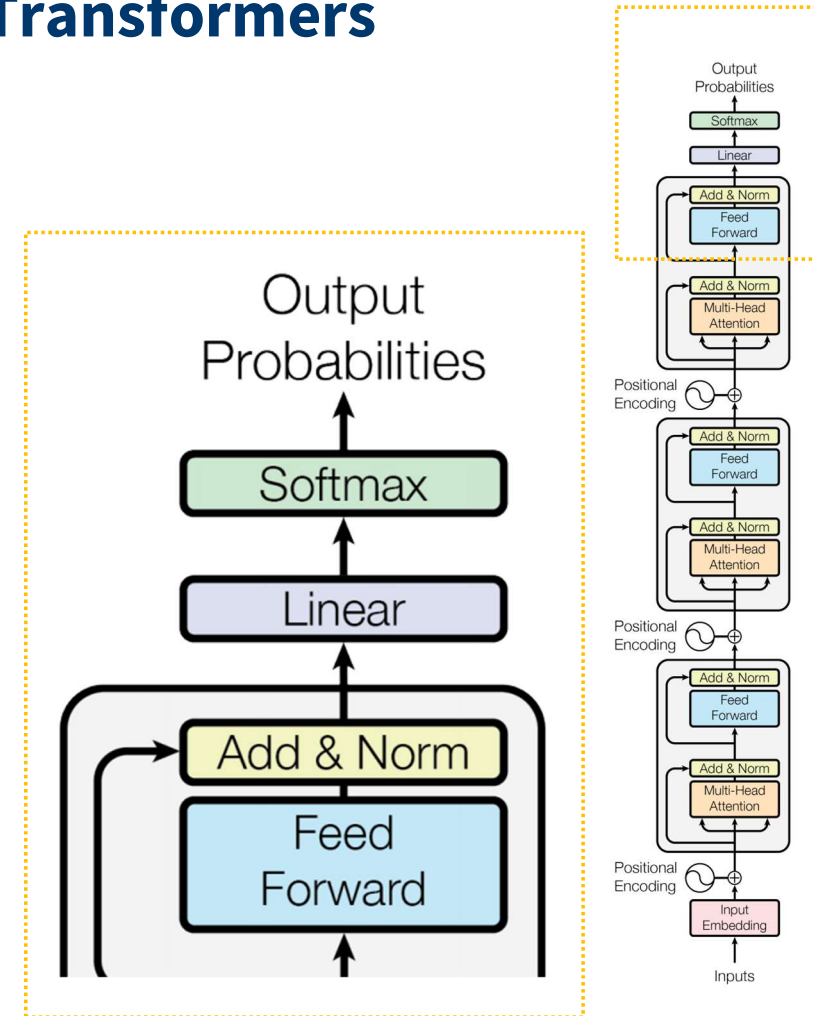
■ Multiple stacked transformer encoder blocks

- BERT Small: 12 layers, 12 heads, 110M parameter
- BERT Large: 24 layers, 16 heads, 340M parameter

■ BERT focuses on text classification

- Linear softmax classifier as output layer
- Typical use cases include sentiment analysis, question answering, NER, NLI, etc.

■ Many extensions: RoBERTa, DistilBERT, ALBERT,...



Adapted from Vaswani et al. (2017)

Bidirectional Encoder Representations from Transformers

Unsupervised pre-training of BERT

■ Trained for two tasks

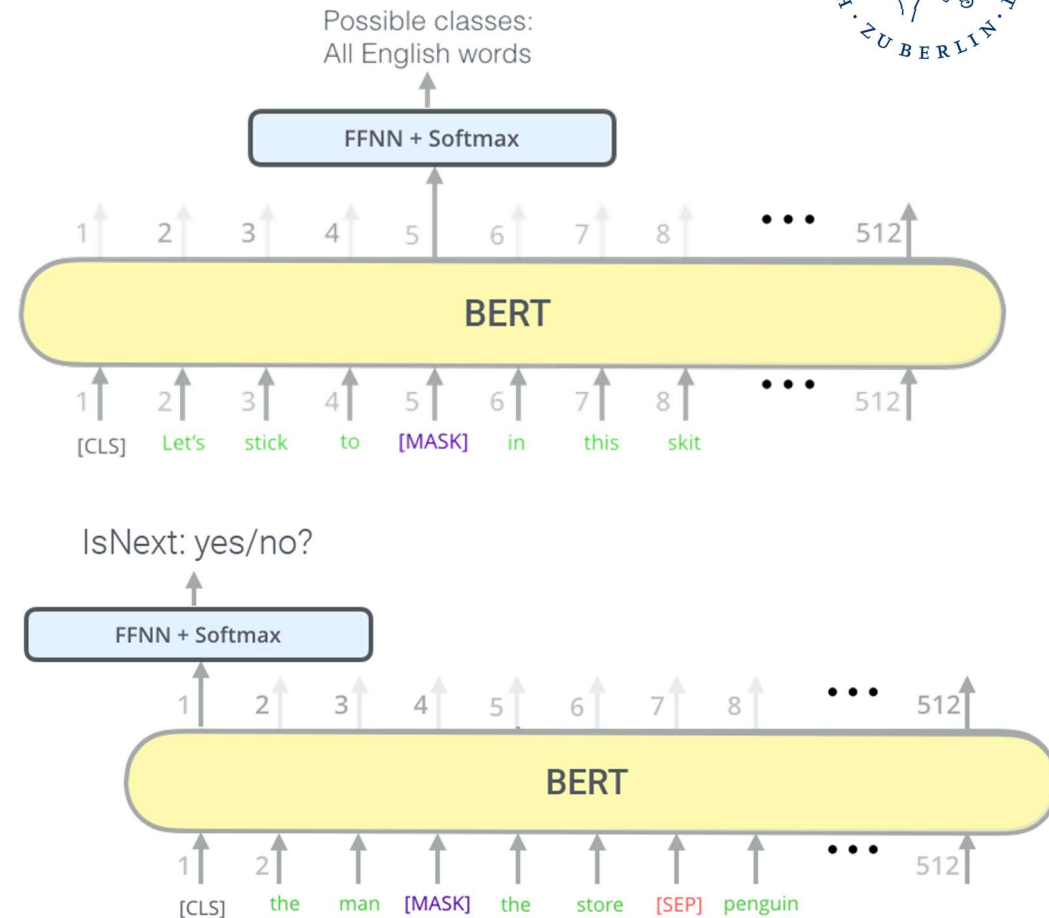
- Masked language model (MLM)
 - Note how MLM fosters bidirectionality
 - Hence the B in BERT
- Next sentence prediction

■ Pre-training data sets

- BooksCorpus (800M words)
- English Wikipedia (2500M words)

■ Finetuning

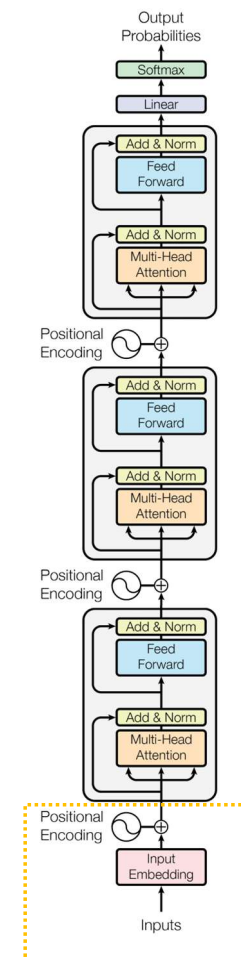
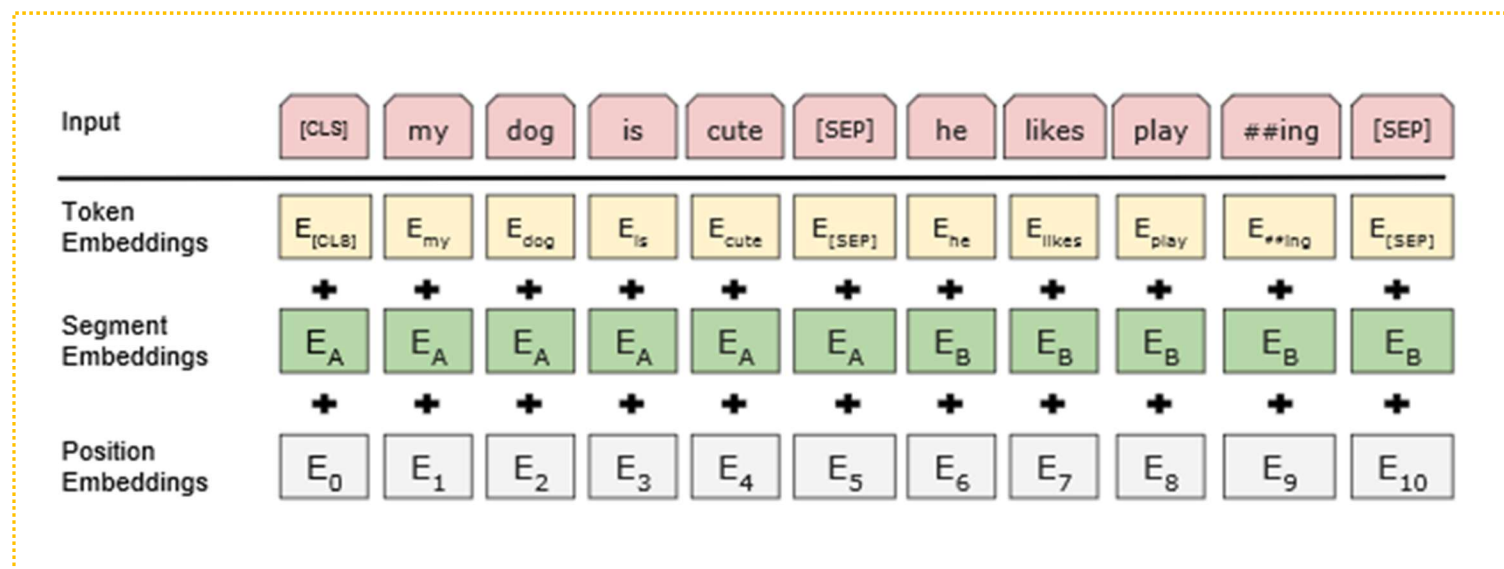
- Using labeled data from downstream task
- Adjust tokenization of data and loss function
- Finetune full-base model with all parameters or just the softmax layer or a mix of both



Bidirectional Encoder Representations from Transformers

Structure of the input data

- Input representation with **token**, **segment**, and **position** embedding
- Specific tokens (CLS, SEP) for pre-training and downstream tasks



Closing Thoughts on Future Studies

■ T5 (Text-To-Text Transfer Transformer)

- Unified Framework: Frames various NLP tasks as text-to-text problems

■ Transformer variants for efficient training & inference

- Longformer, Reformer, BigBird
- Key goal was to handle longer sequences efficiently
- Specific innovations for raising efficiency of finetuning (e.g., adapter modules or Low-Rank Adaptation)

■ Multimodal Transformers

- Vision transformer for image classification
- CLIP and DALL-E combine text and image data for, e.g., text-to-image or image-to-text tasks

■ Advanced training regimes

- Supervised finetuning on high-quality curated data (e.g., ChatGPT 3.5)
- Reinforcement learning from human feedback (RLHF), Direct preference optimization, SteerLM

■ Domain adaptation

- Pre-train on large general corpus, finetune on (large) domain specific corpus for general task, finetune further for specific task
- Basically the ULMFit idea

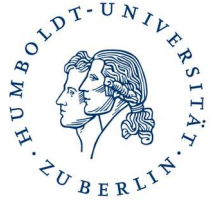
■ Few-, one, and zero-shot learning

■ Prompt engineering



Summary

Summary



Learning goals

- Solid understand of seminar attention is all you need paper
- Self-attention and multi-head attention
- Intuition how popular AI models use the transformer architecture



Findings

- Transformers use self-attention
 - Query, key, value vectors are derived from each token
 - Scaled dot product attention gives output representation
- Multi-head attention
 - Repeat attention mechanism with own Q, K, V matrices
 - Integrate results using properly shaped output matrix
- Positional encoding restore information on token order



What next

- Last lecture session of the semester
- Mainly reserved for Q&A



Homework

- Prepare questions for next session
- What parts of the course would you like to revisit?

Literature



- Alammam, J. (2020). The Illustrated Transformer. Available online: <https://jalammar.github.io/illustrated-transformer/>
- Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. *ArXiv preprint*, arXiv:1409.0473.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems (NIPS'2017)*, Long Beach, CA, USA.
- Zhang, A. Zachary, C., Lipton, M.L., & Smola, A.J. (2023). Dive into Deep Learning. *ArXiv preprint*, arXiv:2106.11342v5. <https://d2l.ai/>

Credits & Disclaimer

- **Course materials have been inspired and draw upon numerous great resources available online**
- **The authors acknowledge the use of and wish to express their gratitude to the following sources**
 - Colah's blog, Understanding LSTM networks (<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>)
 - Zhang et al.: Dive into Deep Learning (<http://d2l.ai/index.html>)
 - MIT 6.S191: Deep Sequence Modeling (<http://introtodeeplearning.com/>)
 - Stanford University: Natural Language Processing with Deep Learning (<https://web.stanford.edu/class/cs224n/>)
 - Oxford University: Deep Natural Language Processing course (<https://github.com/oxford-cs-deepnlp-2017>)

Thank you for your attention!

Stefan Lessmann

Chair of Information Systems
School of Business and Economics
Humboldt-University of Berlin, Germany

Tel. +49.30.2093.5742

Fax. +49.30.2093.5741

stefan.lessmann@hu-berlin.de

<http://bit.ly/hu-wi>

www.hu-berlin.de

